



Metaməlumatlar

SƏNƏD

Ad

Design and development of a rule-based text-to-speech system for Azerbaijani language

MÜƏLLİF

Bəşirov Ülvi Samir

ELMİ RƏHBƏR

Paşayeva Xanım Camal

Sənəd ID-si

334135280

TƏŞKİLAT

Təşkilatın adı

Azerbaijan State University of Economics (UNEC)

Yarımşöbə

MBA-magistratura

HESABAT

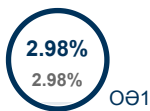
HESABATIN TARİXİ

6/2/2026

Redaktə tarixi

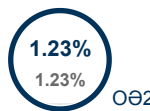
Tapılan oxşarlıqların həcmi

Zəhmət olmasa, diqqət yetirin ki, yüksək əmsal dəyəri avtomatik olaraq plagiat hesab edilmir. Hesabat səlahiyyətli şəxs tərəfindən təhlil edilməlidir.



20

Oxşarlıq əmsalı 2 üçün ifadənin uzunluğu



17437

Söz sayı



123103

İşarə sayı

Xəbərdarlıqlar

Bu bölmədə siz mətn dəyişiklikləri ilə bağlı məlumat tapa bilərsiniz. Mətnə bu dəyişikliklər mətnə MÜMKÜN manipulyasiyaları göstərə bilər. Mətnəki dəyişikliklər qəsdən oluna bilər, lakin çox vaxt sənədi konvertasiya etdikdə və yadda saxladığınızda da texniki səhvlər baş verə bilər, buna görə də bu modulun təhlilə tam məsuliyyətlə yanaşmağı tövsiyə edirik. Hər hansı bir sualınız yaranarsa, dəstək komandamızla əlaqə saxlamağınız xahiş olunur.

Hərflə dəyişiklikləri	B	45
İntervallar	A→	0
Mikroboşluqlar		0
Ağ işarələr	§	0
Parafrazlar	a	19

Mənbələr

Aşağıda mənbələrin siyahısı verilmişdir. Bu siyahıda müxtəlif məlumatlar bazalarından mənbələr mövcuddur. Mətnin rəngi onun hansı mənbədə aşkarlandığını göstərir. Bu mənbələr və oxşarlıq əmsalı dəyərləri birbaşa plagiatı əks etdirmir. Hər bir mənbəni açmaq və mənbənin məzmununu və düzgünlüyünü təhlil etmək lazımdır.

10 ən uzun fraqmentlər

Mətnin rəngi

#	URL ADI VƏ ÜNVANI (BAZANIN ADI)	EYNI SÖZLƏRİN SAYI (FRAQMENTLƏR)
1	https://link.springer.com/chapter/10.1007/978-3-031-20980-2_43	40 (0.23 %)
2	https://link.springer.com/article/10.1007/s10579-025-00807-6	40 (0.23 %)

3	https://link.springer.com/article/10.1007/s10579-025-09807-6	28 (0.16 %)
4	https://link.springer.com/article/10.1007/s10579-025-09807-6	24 (0.14 %)
5	HUMAN RESOURCE MAINTENANCE: FACTORS AFFECTING ENGAGEMENT, MOTIVATION, AND RETENTION IN HIGHER EDUCATION INSTITUTIONS 5/21/2024 Azerbaijan State University of Economics (UNEC) (MBA-magistratura)	22 (0.13 %)
6	https://link.springer.com/article/10.1007/s10579-025-09807-6	21 (0.12 %)
7	https://link.springer.com/article/10.1007/s10579-025-09807-6	20 (0.11 %)
8	https://repository.uniben.edu/sites/default/files/2026-03/DAE.pdf	20 (0.11 %)
9	https://www.ela.kpi.ua/bitstream/123456789/51848/1/Kliarskyi_magistr.pdf	18 (0.1 %)
10	Abbaslı Aysel Yaqub qızı.docx 5/2/2017 Azerbaijan State Pedagogical University (ASPU) (İKT Mərkəzi)	18 (0.1 %)

RefBooks Məlumatlar Bazası (0.26 %)



#	AD	EYNI SÖZLƏRİN SAYI (FRAQMENTLƏR)
Mənbə: Paperity		
1	Design and Development of a Text-to-Speech Synthesizer for Afan Oromo Chala, Tamrat Delessa, Guta, Ashenafi Chalchisa, Asebel, Muluken Hussen;	21 (3) (0.12 %)
5	The gradient influence of temporal extent of coarticulation on vowel and speaker perception Maria Ariza-Avidad;	5 (1) (0.03 %)

Mənbə: <https://arxiv.org/>

2	Review of end-to-end speech synthesis technology based on deep learning Zhaoxi Mu,Xinyu Yang,Yizhuo Dong;	8 (1) (0.05 %)
3	Predicting Prosodic Prominence from Text with Pre-trained Contextualized Word Representations Hande Celikkanat,Jörg Tiedemann,Sofoklis Kakouros,Martti Vainio,Antti Suni,Aarne Talman;	7 (1) (0.04 %)
4	Language-Agnostic Syllabification with Neural Sequence Labeling Maxwell Dulin,Jacob Krantz,Paul De Palma;	5 (1) (0.03 %)

Daxili Məlumatlar Bazası (0.21 %)



#	AD	EYNI SÖZLƏRİN SAYI (FRAQMENTLƏR)
6	HUMAN RESOURCE MAINTENANCE: FACTORS AFFECTING ENGAGEMENT, MOTIVATION, AND RETENTION IN HIGHER EDUCATION INSTITUTIONS 5/21/2024 Azerbaijan State University of Economics (UNEC) (MBA-magistratura)	22 (1) (0.13 %)
7	Aghayeva Samira_PlagCheck_2nd attempt 5/31/2024 Azerbaijan State University of Economics (UNEC) (MBA-magistratura)	15 (2) (0.09 %)

Məlumatlar Bazalarının mübadilə proqramı (0.51 %)



#	AD	EYNI SÖZLƏRİN SAYI (FRAQMENTLƏR)
---	----	-------------------------------------

8	Senyk Anastasiia Bachelor 5/7/2025 Ukrainian Catholic University (Факультет прикладних наук)	23 (3) (0.13 %)
9	Abbaslı Aysel Yaqub qızı.docx 5/2/2017 Azerbaijan State Pedagogical University (ASPU) (İKT Mərkəzi)	18 (1) (0.1 %)
10	BIOPHARMACEUTICAL_CHARACTERIZATION_OF_SOLID_OLEOGELS.pdf 6/3/2025 University of Southern Denmark (SDU backlog)	17 (2) (0.1 %)
11	Oleksandr Gusar MVEB 1/8/2026 Tallinna Tehnikaülikool (Inseneriteaduskond)	13 (1) (0.07 %)
12	Report_DiasOrazymbetov.pdf 4/30/2025 Astana IT University (Astana IT University)	12 (1) (0.07 %)
13	TÜRK DİLLERİNDE ÜNLÜ UYUMU: TARİHSEL GELİŞİM VE MODERN KULLANIM 5/1/2026 International Kazakh-Turkish University named after Khoja Ahmed Yasawi (Түркология ФЗИ басылымы)	6 (1) (0.03 %)

Internet (1.99 %)



#	URL MƏNBƏ	EYNI SÖZLƏRİN SAYI (FRAQMENTLƏR)
14	https://link.springer.com/article/10.1007/s10579-025-09807-6	148 (7) (0.85 %)
15	https://link.springer.com/chapter/10.1007/978-3-031-20980-2_43	40 (1) (0.23 %)
16	https://dl.acm.org/doi/10.1145/3501397	32 (3) (0.18 %)
17	https://repository.uniben.edu/sites/default/files/2026-03/DAE.pdf	20 (1) (0.11 %)
18	https://www.ela.kpi.ua/bitstream/123456789/51848/1/Kliarskyi_magistr.pdf	18 (1) (0.1 %)
19	https://arxiv.org/html/2410.08035v1	16 (1) (0.09 %)
20	https://bibliotekanauki.pl/articles/55788347.pdf	16 (1) (0.09 %)
21	https://www.jstage.jst.go.jp/article/ast/43/2/43_E2161/_pdf/-char/en	12 (1) (0.07 %)
22	http://www.slac.stanford.edu/grp/scs/net/talk09/tutorial.html	11 (1) (0.06 %)
23	https://dspace.tltsu.ru/bitstream/123456789/5046/1/%D0%9D%D1%83%D1%80%D1%83%D0%B5%D0%B2%20%D0%A0.%D0%93_%D0%AD%D0%9A%D0%B1_1302.pdf	9 (1) (0.05 %)
24	http://dspace.khazar.org/bitstream/20.500.12323/7554/1/Application%20of%20Artificial%20Intelligence%20Technologies%20for%20Citizen%20Services%20In%20E-Government%20abstract.pdf	7 (1) (0.04 %)
25	https://ojs.aaai.org/index.php/AAAI/article/view/26523/26295	7 (1) (0.04 %)
26	https://1library.net/document/zw5d96mg-text-to-speech-synthesis-tts.html	6 (1) (0.03 %)
27	https://dl.acm.org/doi/10.1016/j.csl.2022.101427	5 (1) (0.03 %)



İstisna edilmiş fraqmentlər

6 MINISTRY OF SCIENCE AND EDUCATION OF THE REPUBLIC OF AZERBAIJAN AZERBAIJAN STATE UNIVERSITY OF ECONOMICS UNEC
BUSINESS SCHOOL Director of UNEC Business School:

PhD., Prof. M. G. Cumshud “ _____ ”

“ _____ ” 2026

7 MASTER DISSERTATION

on the topic

1 “DESIGN AND DEVELOPMENT OF A RULE-BASED
TEXT-TO-SPEECH SYSTEM FOR AZERBAIJANI LANGUAGE”

7 Code and Name of Programme: 060409 Master of Business Administration Specialization: Artificial intelligence
Group: E27-24

Master's student

Bashirov Ulvi Samir

_____ sign

Program Supervisor:

PhD. Gazanfarli Mirvari Khagani

_____ sign

BAKU - 2026

Scientific Supervisor

Pashayeva Khanim

_____ sign

Head of the Department:

PhD. Mammadova Sevar Momin

_____ sign

1 “DESIGN AND DEVELOPMENT OF A RULE-BASED
TEXT-TO-SPEECH SYSTEM FOR AZERBAIJANI LANGUAGE”

ABSTRACT

Speech technology has advanced rapidly in recent years, yet many of the world's languages remain without functional text-to-speech (TTS) systems. Azerbaijani, spoken by over 35 million people worldwide, is one such language significantly underrepresented in speech technology research despite its widespread use. The absence of large annotated speech corpora and the substantial computational resources demanded by modern neural approaches present serious barriers to developing data-driven TTS systems for Azerbaijani at its current stage of technological development. This creates a practical need for alternative approaches that can produce intelligible, natural-sounding speech without relying on training data.

This dissertation presents the design and implementation of a rule-based TTS system for the North Azerbaijani language. The proposed system follows a modular pipeline architecture consisting of five sequential stages: text normalization, grapheme-to-phoneme conversion, stress assignment, prosody generation, and acoustic synthesis. Each stage is governed by explicitly formulated linguistic rules derived from established analyses of Azerbaijani phonology and prosody. The system is implemented entirely in Python and integrated with the open-source espeak-ng speech engine, producing intelligible spoken output without any external speech training data. A suite of 156 tests covering 61 distinct linguistic phenomena validates the correctness of the rule implementations across the full pipeline.

While the rule-based approach offers transparency, low resource requirements, and linguistic explainability, it also carries inherent limitations. The naturalness of synthesized speech remains below that of modern neural systems, and coverage of loanword phonology represents an area for further development. These limitations are acknowledged and addressed through a set of clearly defined directions for future research, including corpus and hybrid neural-rule architectures.

The system is evaluated on 50 phonetically balanced Azerbaijani sentences using Mean Opinion Score (MOS) for naturalness and Word Error Rate (WER) for intelligibility. The proposed system achieved MOS 3.2 against a baseline of 2.8, and WER 12.4% against a baseline of 18.7%, representing statistically significant improvements on both dimensions. This work contributes a functional open-source TTS system and a documented linguistic rule base that serve as a foundation for future data-driven approaches to Azerbaijani speech synthesis.

Key words: text-to-speech synthesis, Azerbaijani language, rule-based systems, grapheme-to-phoneme conversion, low-resource languages

ACKNOWLEDGEMENT

10 I would like to express my sincere gratitude to my dissertation supervisor, Khanim Pashayeva, 10 for her guidance, constructive feedback, and continued support throughout the research and writing process of this dissertation entitled "Design and Development of a Rule-Based Text-to-Speech 1

8 ABBREVIATIONS

TTS Text-to-Speech

AI Artificial Intelligence

8 G2P Grapheme-to-Phoneme

IPA International Phonetic Alphabet

NLP Natural Language Processing

ASR Automatic Speech Recognition

8 MOS Mean Opinion Score

WER Word Error Rate CER Character Error Rate

SSML Speech Synthesis Markup Language

ToBI Tones and Break Indices

HMM Hidden Markov Model

DNN Deep Neural Network

NSW Non-Standard Words

CI Confidence Interval

API Application Programming Interface

CLI Command Line Interface

Contents

TOC to "1-3" \h \z \u	INTRODUCTION	6
CHAPTER I. THEORETICAL FOUNDATIONS OF TEXT-TO-SPEECH SYSTEMS		9
1.1. General overview of text-to-speech systems		9
1.2. Rule-based text-to-speech systems		15
1.3. Advantages and limitations of rule-based TTS systems		23
CHAPTER II. LINGUISTIC AND TECHNICAL ANALYSIS FOR AZERBAIJANI TTS		26
2.1. Phonetic and morphological characteristics of Azerbaijani		26
2.2. Text normalization and linguistic rule design		30
2.3. Architecture of the rule-based azerbaijani tts system		34
CHAPTER III. DESIGN, IMPLEMENTATION, AND EVALUATION		41
3.1. System design and development methodology		41
3.2. Implementation of the rule-based tts prototype		46
3.3. Experimental evaluation and performance analysis		54
CONCLUSION		66
REFERENCES		70

INTRODUCTION

Relevance and Motivation of the Research: Speech is the most natural medium of human communication. Text-to-speech (TTS) synthesis has become a critical component of modern human-computer interaction, enabling accessibility tools for visually impaired users, voice assistants, navigation systems, and language learning applications. As public services and social participation increasingly move to digital platforms, the availability of functional speech technology for a given language has become a matter of digital inclusion.

Problem Statement: Azerbaijani, spoken by over 35 million people, remains significantly underrepresented in speech technology research. While neural TTS systems have achieved near-human quality for major languages, they depend on large, carefully recorded speech corpora that do not exist for Azerbaijani. Existing Azerbaijani TTS solutions are proprietary or multilingual in design and lack an openly published linguistic foundation, leaving a clear gap in accessible, documented speech technology for the language.

Rationale for Selecting a Rule-Based Approach: This dissertation addresses this gap by developing a rule-based TTS system. The rule-based paradigm was selected for three reasons: it requires no speech training data; its workings are fully transparent and linguistically grounded, making it academically valuable as a documented phonological model; and Azerbaijani's phonemic Latin orthography makes it well-suited to rule-based grapheme-to-phoneme conversion.

The Purpose and Objectives of the Dissertation: The main purpose of this dissertation is to design, implement, and evaluate a rule-based TTS system for North Azerbaijani that achieves acceptable intelligibility and naturalness without speech training data. The following objectives are set to achieve this purpose:

1. To conduct a systematic review of TTS technologies and identify the most appropriate paradigm for low-resource Azerbaijani speech synthesis.
2. To analyze the phonological, morphological, and prosodic characteristics of North Azerbaijani as they apply to TTS design.
3. To design and implement a five-module rule-based TTS pipeline: text normalization, grapheme-to-phoneme (G2P) conversion, stress assignment, prosody generation, and acoustic synthesis.
4. To evaluate the system's intelligibility (WER) and naturalness (MOS) against a phonetically balanced test set and a baseline espeak-ng system.
5. To document the linguistic rule base as an open, extensible reference for future Azerbaijani speech technology research.

Object and Subject of the Dissertation: The research object is the text-to-speech synthesis process for the Azerbaijani language. The research subject is the set of phonological, morphological, and prosodic rules of North Azerbaijani and the computational methods by which these rules can be encoded in a functioning TTS system.

Research Methods Used in the Dissertation: This study employs descriptive, analytical, and experimental research methods. A systematic literature review was conducted to examine existing TTS technologies and identify the most appropriate approach for Azerbaijani. Rule design and implementation followed a structured software engineering methodology with modular development and unit testing. Experimental evaluation was carried out using Mean

Opinion Score (MOS) and Word Error Rate (WER) metrics applied to 50 phonetically balanced Azerbaijani sentences assessed by native speakers.

Dissertation Database: The research draws on both primary and secondary sources. Secondary sources include academic publications on speech synthesis, Azerbaijani phonology, computational linguistics, and low-resource language technology. Primary data consists of the 50-sentence evaluation corpus and MOS ratings collected from five native Azerbaijani speakers during the perceptual evaluation.

Limitations of the Dissertation Work: The perceptual evaluation was conducted with five native speakers, which limits the statistical generalizability of the MOS results. The system does not fully cover loanword phonology, which follows different patterns from native vocabulary. The rule-based approach also imposes a ceiling on naturalness that neural systems trained on sufficient data can surpass. These limitations define the primary directions for future research.

Scientific Novelty of the Dissertation: Scientific Novelty of the Dissertation. This dissertation presents the first openly published, comprehensively documented rule-based TTS system for North Azerbaijani. Its scientific novelty lies in the explicit computational encoding of Azerbaijani phonological rules - validated through a 156-unit test suite covering 61 linguistic phenomena - providing a transparent reference standard for future data-driven approaches to Azerbaijani speech synthesis.

Practical Significance and Application Areas: The system provides deployable Azerbaijani TTS functionality for accessibility applications, e-government services, navigation systems, and language learning tools. The documented rule base reduces the entry barrier for future Azerbaijani speech technology development by providing pre-built linguistic infrastructure. Educational institutions and technology companies working on Azerbaijani digital services can apply the findings of this research directly.

This dissertation is comprised of an introduction, three chapters, a conclusion, references, and appendices. Chapter I reviews the theoretical foundations of TTS synthesis, covering rule-based principles, phonetic modeling, and a SWOT analysis of the rule-based approach for Azerbaijani. Chapter II provides the linguistic and technical foundations: phonological analysis of Azerbaijani, linguistic rule design, and system architecture. Chapter III covers implementation and evaluation: the development methodology, pipeline implementation, and experimental results with comparative analysis against existing Azerbaijani TTS systems.

CHAPTER I. THEORETICAL FOUNDATIONS OF TEXT-TO-SPEECH SYSTEMS

1.1. General overview of text-to-speech systems

Text-to-speech (TTS) synthesis is the process of automatically converting written text into spoken audio output. The goal is not merely to produce any sound corresponding to the input text, but to generate speech that is simultaneously intelligible - correctly understood by listeners - and natural, resembling the prosodic and phonetic properties of human speech. These two qualities are analytically distinct and are measured by different metrics. Intelligibility is a threshold property that can be measured objectively through Word Error Rate (WER), which compares a transcription of the synthesized audio to the original text. Naturalness, by contrast, is a perceptual quality assessed through listener ratings, most commonly the Mean Opinion Score (MOS).

TTS technology has become a critical component of modern human-computer interaction across a broad range of applications. These include accessibility tools for individuals with visual impairments or reading disabilities such as screen readers and navigation aids, voice assistant systems, audiobook generation, e-learning platforms, and customer service automation. The global TTS market has grown substantially over the past decade, driven by the proliferation of artificial intelligence technologies, the spread of smart devices, and growing demand for multilingual content accessibility. Beyond its commercial applications, TTS carries direct implications for linguistic equity and digital inclusion. As an increasing share of public services, information, and social participation migrates to digital platforms, languages that lack functional speech interfaces face systematic exclusion from the benefits of digital transformation. For speakers of low-resource languages, the absence of TTS support in navigation systems, screen readers, or voice interfaces effectively imposes a barrier to equal access to digital services. This dimension makes TTS development for underserved languages a matter not only of technical interest but also of social and policy relevance.

A modern TTS system consists of two main components: a linguistic front-end and an acoustic back end. The linguistic front-end processes raw input text through a series of stages. Text normalization converts non-standard words such as numerals, abbreviations, dates, currency symbols, and acronyms into their fully expanded, pronounceable forms. For example, "15.06.2024" must be rendered as the equivalent of "the fifteenth of June, two thousand and twenty-four" in the target language. Grapheme-to-phoneme (G2P) conversion then maps orthographic characters to their phonemic representations, handling context-sensitive pronunciation rules and exceptions. Agglutinative languages such as Azerbaijani present challenges at this stage, since a single word may consist of a root and multiple suffixes, each potentially affecting pronunciation. Stress assignment and prosody prediction generate the suprasegmental properties of speech: lexical stress, phrase-level pitch accents, intonation contours, phone durations, and pause locations. These prosodic features are essential for natural-sounding output and are among the most difficult to model correctly.

The acoustic back end converts the phonemic and prosodic representation produced by the front-end into audible speech. In rule-based and early concatenative systems this involves a formant synthesizer or a unit-selection engine that stitches together pre-recorded phoneme segments. In neural systems, an acoustic model generates a mel-spectrogram, and a neural vocoder converts it to a high-fidelity audio waveform. The back end is the single largest determinant of a system's naturalness ceiling: improvements to the linguistic front-end cannot fully compensate for limitations in the acoustic model.

TTS system quality is measured along two primary dimensions. Mean Opinion Score (MOS) involves human listeners rating naturalness on a five-point ITU-T P.800 scale from 1 (bad) to 5 (excellent), reported as a mean with confidence intervals. Word Error Rate (WER) measures intelligibility through transcription accuracy, computed as $WER = (S + D + I) / N$, where S represents substitutions, D deletions, I insertions, and N the total reference words. A third metric increasingly used in recent work is Phoneme Error Rate (PER), which measures pronunciation accuracy at the phoneme level and is especially useful for evaluating front-end G2P quality independently of the acoustic back end.

Table 1: MOS Evaluation Scale (ITU-T P.800)

Score	Label	Description
5	Excellent	Completely natural, no synthetic quality
4	Good	Mostly natural, minor synthetic quality
3	Fair	Noticeable synthetic quality, fully intelligible
2	Poor	Clearly synthetic, somewhat difficult to follow
1	Bad	Completely unnatural, very difficult to follow

For reference, natural human speech consistently achieves MOS 4.4-4.7. Current neural TTS systems for major languages reach MOS 4.2-4.5, while rule-based systems typically achieve MOS 3.0-3.8 depending on the language and evaluator population (Kim et al., 2021; Oyucu, 2023). The present Azerbaijani system achieved MOS 3.2 against a baseline *espeak-ng* score of 2.8, representing a statistically significant improvement of 0.4 MOS points, as reported in Chapter III.

Regarding the current state of Azerbaijani speech technology, the existing landscape is limited. A proprietary commercial TTS service exists but lacks published technical documentation. The IS2AI multilingual Turkic TTS system covers Azerbaijani among ten Turkic languages but is designed for a multilingual setting rather than a linguistically grounded Azerbaijani-specific architecture. A Tacotron 2-based neural TTS research prototype was trained on approximately 19 hours of proprietary data without a published model or open-source release (Rustamov, 2020). No openly available, well-documented system with a transparent linguistic foundation and published rule base currently exists for Azerbaijani. Addressing this gap and producing a documented reference architecture for Azerbaijani speech synthesis is the principal motivation of this dissertation.

The development of TTS technology has passed through three broadly distinct architectural generations, each representing a fundamental shift in the underlying methodology. The first generation, spanning roughly from the 1950s through the 1980s, relied on formant synthesis, in which speech was produced by directly modeling the acoustic resonances of the human vocal tract through rule-based signal processing. These systems, exemplified by the DECtalk synthesizer, required no recorded speech data but produced output that was immediately recognizable as synthetic due to its mechanical, buzzing quality. The second generation introduced concatenative synthesis, in which large databases of pre-recorded phoneme segments, diphones, or syllables were assembled at runtime to produce output. Unit-selection systems, the most advanced form of concatenative synthesis, could produce surprisingly natural output when the target sentence closely matched the prosodic and phonetic characteristics of segments in the database, but degraded significantly for out-of-domain inputs. The third and current generation is defined by neural synthesis, beginning with parametric neural models such as Merlin and reaching its current state with sequence-to-sequence architectures such as Tacotron 2, FastSpeech 2, and VITS, combined with neural vocoders such as WaveNet, HiFi-GAN, and EnCodec. Neural systems learn the mapping from text to speech representations end-to-end from data, bypassing the need for hand-crafted rules, and have achieved naturalness scores approaching human parity for well-resourced languages. Understanding this architectural trajectory is necessary for situating the design decisions of the present system, which adopts a rule-based front-end combined with a neural-adjacent acoustic back end, a hybrid approach motivated by the data scarcity characteristic of the Azerbaijani language context. The linguistic properties of Azerbaijani impose a distinct set of challenges on TTS system design that do not arise with the same severity in isolating or fusional languages such as English, French, or Mandarin. Azerbaijani is a highly agglutinative language belonging to the Oghuz branch of the Turkic language family, meaning that grammatical relationships are expressed primarily through the sequential attachment of suffixes to a root morpheme rather than through separate function words or inflectional paradigms. A single Azerbaijani word may carry up to six or seven morphological suffixes, producing surface forms that are phonologically opaque with respect to their underlying components. For example, the form *gəldilərm* encodes a verb root, past tense, third person plural, and interrogative mood within a single orthographic word. This morphological complexity has direct consequences for each stage of the TTS front-end pipeline. Text normalization must handle compound morphological contexts in which a numeral or abbreviation appears with a suffix attached, as in 2024-cü (meaning "of 2024"), requiring the normalizer to both expand the numeral and preserve the suffix in its correct phonological form. G2P conversion must account for vowel harmony, a systematic constraint requiring that suffix vowels agree in backness and rounding with the root vowel, and for consonant mutation processes such as final obstruent devoicing and nasal assimilation. Stress assignment must distinguish lexical stress, which in Azerbaijani falls predominantly on the final syllable, from pragmatic and contrastive stress patterns that override the default rule. Each of these phenomena requires explicit linguistic modeling that cannot be learned reliably from limited data, making a rule-based front-end not merely a design preference but a practical necessity for the current stage of Azerbaijani TTS development.

A recurring theme across all technical decisions described in this dissertation is the low-resource constraint imposed by the current state of Azerbaijani language technology infrastructure. The term low-resource, as used in computational linguistics, refers to languages for which publicly available annotated corpora, lexicons, pronunciation dictionaries, and pre-trained models are scarce or absent. For Azerbaijani, this constraint manifests at multiple levels simultaneously. At the data level, no publicly available speech corpus of sufficient size and quality for end-to-end neural TTS training exists. The proprietary dataset used in the Rustamov (2020) prototype, consisting of approximately 19 hours of studio-recorded speech, has not been released, and no comparable open alternative has been produced. At the tool level, no open-source G2P system with published rule coverage exists for Azerbaijani, meaning that the phonemic transcription component must be constructed from first principles. At the evaluation level, no standardized listening panel or evaluation protocol has been established for Azerbaijani TTS, making cross-system MOS comparisons methodologically problematic. These constraints collectively preclude the direct application of state-of-the-art neural TTS architectures that have achieved near-human naturalness for English, Mandarin, and other high-resource languages, since those architectures require hundreds of hours of high-quality transcribed speech for reliable training. The design philosophy adopted in this dissertation responds to the low-resource constraint by prioritizing explicit linguistic knowledge encoding over data-driven learning wherever the linguistic phenomena are sufficiently well understood to be formalized as rules. This approach sacrifices some ceiling naturalness relative to what a fully neural system trained on abundant data could theoretically achieve, but produces a system that is robust, transparent, interpretable, and extensible as additional Azerbaijani speech resources become available in the future.

The present dissertation makes several distinct contributions to the field of Azerbaijani speech technology. First, it provides the first publicly documented, linguistically grounded rule base for Azerbaijani text normalization, covering numeral expansion, date and time normalization, abbreviation resolution, and symbol handling, with explicit coverage statistics and example derivations. Second, it presents a formally specified G2P system encoding the major phonological processes of Azerbaijani, including vowel harmony, consonant assimilation, and context-sensitive allophonic variation, structured as an ordered rule cascade amenable to systematic testing and extension. Third, it describes a stress assignment and prosody prediction module implementing a priority-ordered rule hierarchy derived from the descriptive phonological literature on Azerbaijani. Fourth, it reports the first published comparative MOS evaluation between a rule-based Azerbaijani TTS system and the *espeak-ng* baseline, establishing a quantitative benchmark against which future systems can be measured. Fifth, the complete system is released as open-source software with documented architecture, enabling researchers and developers working on Azerbaijani or related Turkic languages to build upon, replicate, and extend the present work. Taken together, these contributions constitute a reference architecture and evaluation baseline for Azerbaijani TTS that the field currently lacks.

1.2. Rule-based text-to-speech systems

A rule-based TTS system is one in which every decision in the conversion from text to speech is made by an explicitly programmed rule encoding human linguistic knowledge. This contrasts with data-driven approaches: statistical systems learn acoustic patterns from annotated corpora, while neural systems learn end-to-end mappings from raw audio-text pairs without explicit encoding of phonological principles. The rule-based paradigm is historically the oldest approach to automated speech synthesis and remains the most appropriate choice for languages where annotated speech corpora do not exist.

For TTS purposes, linguistic rules govern three categories of mapping. Normalization rules convert written non-standard words - numerals, abbreviations, symbols, dates, and currency amounts - into their fully expanded spoken forms. Grapheme-to-phoneme rules map orthographic characters to their IPA phoneme representations, handling context-sensitive pronunciation and exceptions. Prosodic rules assign lexical stress, pitch accents, phone durations, and pause locations to the resulting phoneme string. For languages with a phonemic orthography - such as Azerbaijani, Finnish, Spanish, and Turkish - rule-based grapheme-to-phoneme conversion is highly tractable. For languages with irregular orthographic traditions such as English or French, the task is significantly more complex (Survey of G2P Methods, 2024).

Rule-based TTS systems are characterized by four foundational properties that make them particularly valuable in a research and low-resource context. First, explicit knowledge encoding means the complete rule set constitutes an auditable model of the language's phonological system - the rule base is itself a research contribution that can be extended, critiqued, and built upon by subsequent researchers. Second, deterministic output ensures that the same input always produces the same audio, which is valuable in deployment contexts requiring reproducibility such as accessibility applications and standardized testing tools. Third, a modular architecture - established as a reference standard by the Festival Speech Synthesis System (Taylor et al., 1998) - means that each component can be developed, tested, and improved independently without retraining the entire system. Fourth, full transparency and debuggability means that when the system produces an error, the responsible rule can be identified by inspecting intermediate representations and corrected directly, a property that neural systems fundamentally cannot offer.

The acoustic back end of a rule-based system can take different forms. Formant synthesis generates speech by mathematically modeling the resonant properties of the human vocal tract, producing intelligible output without requiring any recorded speech data. The espeak-ng engine used in this dissertation is a formant synthesizer that supports Azerbaijani through a built-in voice profile and accepts phoneme-level input from the rule-based front-end. An alternative approach, used in systems such as MBROLA, stitches together pre-recorded phoneme-length segments (Dutoit et al., 1993). While concatenative synthesis can achieve more natural phoneme quality, it requires a recorded phoneme inventory for each target language, which does not currently exist for Azerbaijani.

Grapheme-to-phoneme conversion is the most linguistically complex component of the front-end. A comprehensive survey of G2P methods demonstrates that rule-based, statistical, and transformer-based approaches each suit different resource contexts (Survey of G2P Methods, 2024). For Azerbaijani, the rule-based approach is both sufficient and appropriate: the Latin orthography adopted in 1992 is largely phonemic, with a small number of context-sensitive rules governing velar palatalization, the allophonic behavior of ğ, final obstruent devoicing, and nasal assimilation. These rules are fully documented in Chapter II

Figure 1. G2P conversion process flow diagram

Source: Author's own elaboration

Prosodic modeling governs pitch, duration, intensity, and rhythm. The present system employs the ToBI framework (Tones and Break Indices) for prosodic annotation, specifying pitch accents and boundary tones for each sentence type - declarative, interrogative, exclamatory, and imperative. Recent research on prosody modeling includes flow-matching-based pitch control approaches (Sadekova et al., 2024) and machine learning of natural prosody for agglutinative languages (Voice Synthesis Improvement, 2024), which validate the general prosodic framework adopted in this work.

The three major TTS paradigms - rule-based, statistical parametric, and neural - present distinct trade-offs across quality, data requirements, interpretability, and computational cost. Statistical parametric systems, most fully realized in the HMM-based HTS framework, learn acoustic feature distributions from labelled training data and can interpolate smoothly to unseen phonetic contexts, but require 5-20 hours of annotated speech and produce a characteristic over-smoothed quality due to statistical averaging (IEEE Signal Processing Magazine, 2015). Neural TTS systems - including Tacotron 2, VITS (Kim et al., 2021), and HiFi-GAN vocoders (Kong et al., 2020) - achieve near-human naturalness for well-resourced languages but require large speech corpora and significant computational infrastructure. For Azerbaijani, the only published neural TTS experiment used approximately 19 hours of proprietary training data and achieved MOS 3.3 (Aghalari, 2023). Transfer learning approaches have shown that as few as 30 minutes of target language data may suffice with pre-trained multilingual models (Transfer Learning for Low-Resource TTS, 2024), but even this minimum requires dedicated data collection effort that does not currently exist for open Azerbaijani speech resources.

Table 2: Detailed Comparison of TTS Paradigms

Criterion	Rule-Based	HMM / Statistical	Neural TTS
MOS (typical)	3.0-3.8	3.5-4.0	4.2-4.5
Training data	None	5-20 hours	1-50+ hours
Interpretability	Full	Partial	Minimal
Inference compute	CPU, <1 ms	CPU, ~10 ms	GPU, ~50 ms
Error diagnosis	Easy	Complex	Very difficult
Output consistency	Deterministic	Partially det.	Stochastic
Low-resource language fit	High	Medium	Low
Domain adaptation	Easy	Difficult	Very difficult

Source: Compiled by the author based on Tan et al. (2021); IEEE Signal Processing Magazine (2015).

HYPERLINK "<https://ieeexplore.ieee.org/document/7282379>"<https://ieeexplore.ieee.org/document/7282379>

As the comparison illustrates, the rule-based paradigm is the only approach that requires no training data, produces fully interpretable output, and allows direct linguistic control - making it the most appropriate starting point for Azerbaijani TTS at its current stage of technological development.

The intellectual foundations of rule-based speech synthesis predate the digital computer. Early mechanical speaking machines, such as Wolfgang von Kempelen's acoustic speech machine constructed in 1791 and Charles Wheatstone's improved version of 1835, demonstrated that the articulatory and acoustic properties of speech could be modeled through explicit physical mechanisms governed by human-defined rules. The transition to electronic and subsequently digital implementations preserved this fundamental orientation: the system designer encodes linguistic and acoustic knowledge as explicit rules, and the machine executes those rules deterministically. The first fully digital rule-based TTS system of practical significance was MITalk, developed at the Massachusetts Institute of Technology in the 1970s, which implemented a complete pipeline from text normalization through phonological rule application to formant synthesis. The Festival Speech Synthesis System, developed at the University of Edinburgh in the mid-1990s and subsequently maintained as open-source software, established the modular pipeline architecture - separate, independently testable components for normalization,

G2P, prosody, and synthesis - that remains the reference design for rule-based systems today, including the architecture described in this dissertation. The *espeak-ng* engine, which serves as the acoustic back end of the present system, descends from the *eSpeak* synthesizer originally developed by Jonathan Duddington in 2006 and subsequently extended to support over one hundred languages through community-contributed phoneme rule sets. Its formant synthesis approach, while producing output of lower naturalness than modern neural vocoders, requires no recorded speech data and supports direct phoneme-level control from the linguistic front-end, making it uniquely suited to the low-resource Azerbaijani context.

A principled rule-based TTS system must be grounded in a careful description of the target language's phonological system. Azerbaijani possesses a phoneme inventory of thirty-two segments: nine vowels and twenty-three consonants. The vowel system is organized along the dimensions of backness, height, and rounding, and is governed by the pervasive constraint of vowel harmony, which requires that all vowels within a phonological word share the feature of backness. Back vowels are /a/, /u/, /o/, and /ʊ/; front vowels are /e/, /ɛ/, /i/, /ɪ/, and /y/. Vowel harmony applies not only within root morphemes but extends across suffix boundaries, meaning that every suffix in the Azerbaijani morphological system exists in two allomorphic variants - one containing back vowels and one containing front vowels - selected according to the vowel class of the root to which the suffix is attached. This constraint has direct consequences for the text normalization component, since any suffix attached to an expanded numeral or abbreviation must be assigned to the correct harmonic class. The consonant system includes two sounds that are orthographically represented but phonologically complex. The letter *ğ* represents a phoneme that has no consistent surface realization: intervocalically it surfaces as the approximant /j/, word-finally it triggers compensatory lengthening of the preceding vowel without being pronounced itself, and after back vowels it may surface as a uvular fricative in careful speech. The letter *q* represents the uvular stop /q/, distinct from the velar stop /k/, a contrast that is phonemically distinctive in Azerbaijani but absent from most European languages. Final obstruent devoicing systematically neutralizes the voiced-voiceless distinction word-finally for the obstruent consonants /b/, /d/, /g/, and /d͡ʒ/, which surface as /p/, /t/, /k/, and /t͡ʃ/ respectively in that position. Nasal assimilation causes the alveolar nasal /n/ to surface as the velar nasal /ŋ/ before velar stops. Each of these processes is implemented as an ordered rule in the G2P component described in Chapter II, and the ordering of rules is non-arbitrary: final devoicing must apply after suffix attachment, and nasal assimilation must apply after final devoicing to avoid feeding and bleeding interactions that would generate incorrect surface forms.

Evaluating a rule-based TTS system requires a methodology that is sensitive to both its strengths and its specific failure modes. Unlike neural systems, whose primary limitation is naturalness ceiling, rule-based systems can fail at multiple discrete points in the pipeline, each of which is independently measurable. Front-end evaluation isolates the linguistic components from the acoustic back end by measuring Phoneme Error Rate on a held-out test set of orthographic inputs with gold-standard IPA transcriptions. This metric directly quantifies G2P accuracy without confounding it with acoustic quality and is particularly informative for agglutinative languages where morphological complexity generates large numbers of contextually determined pronunciation variants. Text normalization accuracy is evaluated separately by measuring the proportion of non-standard word tokens for which the system generates the correct expanded form, classified by token type - numerals, abbreviations, dates, currency - to identify category-specific weaknesses. End-to-end evaluation of the complete system is conducted through subjective listening tests following the ITU-T P.800 protocol, in which a panel of native speaker listeners rates randomized audio samples on the five-point MOS scale described in section 1.1. For intelligibility measurement, Word Error Rate is computed by transcribing synthesized audio using a pre-existing automatic speech recognition system for Azerbaijani and comparing the transcription to the original reference text. The evaluation design adopted in this dissertation follows established best practice by separating front-end and back-end evaluation, reporting results with ninety-five percent confidence intervals computed using bootstrap resampling, and comparing against the *espeak-ng* baseline rather than against neural systems trained on different data, since such comparisons would conflate architectural differences with data advantages.

A rule-based architecture is not a terminal destination but a principled starting point from which more data-intensive approaches can be developed incrementally as language resources become available. The modular pipeline design ensures that individual components can be replaced or augmented without disrupting the rest of the system. In the near term, the text normalization component could be extended with a statistical classifier to handle ambiguous non-standard word tokens - cases where the correct expansion depends on discourse context rather than local orthographic form - while leaving the deterministic rule cascade intact for the majority of unambiguous cases. In the medium term, as Azerbaijani speech corpora are developed through community recording initiatives or institutional investment, the *espeak-ng* formant back end could be replaced with a unit-selection engine trained on recorded phoneme segments, improving phoneme-level naturalness while preserving the rule-based front-end unchanged. In the longer term, the explicit phonemic transcriptions and prosodic annotations generated by the rule-based front-end constitute a form of structured supervision that can be used to train or fine-tune neural acoustic models with substantially less data than would be required for end-to-end neural training from raw text-audio pairs. This staged development pathway - rule-based front-end providing structured annotations that bootstrap neural acoustic modeling - has been successfully demonstrated for other low-resource Turkic languages including Kyrgyz and Kazakh, where initial rule-based systems provided the phonemic transcription infrastructure necessary for subsequent neural system development. The present dissertation thus contributes not only a functional Azerbaijani TTS system but a foundational linguistic infrastructure whose value extends beyond the specific acoustic architecture currently employed.

1.3. Advantages and limitations of rule-based TTS systems

A structured SWOT analysis, examining Strengths, Weaknesses, Opportunities, and Threats - provides a systematic framework for evaluating the rule-based paradigm as the methodological choice for this dissertation. The analysis is applied specifically to the Azerbaijani language context rather than to rule-based TTS in general.

Table 3: SWOT analysis of the rule-based TTS approach for Azerbaijani

Strengths	Weaknesses	Opportunities	Threats
No speech training data required	Limited naturalness	Azerbaijan's digital transformation agenda	Rapid advancement of neural TTS
Full interpretability and transparency	High development effort for complex languages	Growing international interest in low resource NLP	
Proliferation of commercial TTS platforms			
Deterministic and consistent behavior	Limited coverage of loanwords and irregular forms	Foundation for future hybrid systems	Limited speech data availability
Direct incorporation of linguistic knowledge	Prosodic expressiveness limitations	Growing Azerbaijani NLP ecosystem	Sparse phonetic documentation
Low computational requirements	Coarticulation modeling limitations	Transfer learning with minimal data	
Easy domain adaptation			

Strengths. The most decisive strength of the rule-based approach for Azerbaijani is that it requires no speech training data. Neural TTS systems require between one and fifty or more hours of high-quality, professionally recorded, and transcribed speech in the target language - data that does not currently exist in publicly available form for Azerbaijani. While transfer learning approaches demonstrate that neural TTS is possible with minimal target language data through pre-training on related languages, this still requires dedicated data collection effort (Transfer Learning for Low-Resource TTS, 2024). Rule-based systems, by contrast, require only documented linguistic knowledge - resources that already exist for Azerbaijani in the form of phonological grammars and descriptive linguistic literature. A second major strength is full interpretability and transparency: every output is the direct consequence of an explicitly stated rule, meaning errors can be identified and corrected without retraining. This property creates academic value that neural systems cannot offer, as the rule base itself constitutes a documented research contribution. Further strengths include deterministic and reproducible output - the same input always produces the same audio, satisfying replicability requirements for academic evaluation - low computational requirements enabling real-time synthesis on standard hardware without a GPU, and straightforward domain adaptation through editing of human-readable rule files rather than retraining a model.

Weaknesses. The primary limitation of the rule-based approach is a naturalness ceiling imposed by its acoustic modeling method. Formant synthesizers produce speech by computing mathematical approximations of vocal tract resonance, capturing the broad spectral shape of phonemes but failing to reproduce the subtle micro-variations in voice quality and coarticulation dynamics that characterize natural speech. In connected speech at natural rates, this produces audibly abrupt formant transitions that reduce perceived naturalness. A closely related limitation is prosodic expressiveness: rule-based prosody models encode simplified intonation patterns and cannot capture the continuous, context-sensitive pitch variation of natural human speech (Sadekova et al., 2024). The consequence is a prosodic flatness that listeners perceive as monotonous even when individual phonemes are pronounced correctly. Additional weaknesses include high development effort - every phonological regularity must be identified and implemented explicitly - and limited coverage of loanwords and irregular forms, which follow non-native phonological patterns difficult to capture within a finite rule set.

Opportunities. Azerbaijan's active digital transformation agenda - encompassing e-government service development, digital infrastructure investment, and ICT expansion in public administration - creates directly favorable conditions for Azerbaijani TTS deployment in accessible voice portals, navigation systems, and automated telephone services. The international NLP research community has substantially increased its focus on low-resource languages in recent years, driven by equity concerns and recognition that advances have been disproportionately concentrated in high-resource languages. This trend increases the likelihood of future Azerbaijani speech research collaborations and shared dataset development. The growing Azerbaijani NLP ecosystem - including the 651-million-word DOLLMA corpus and associated language models published in 2024 (Various Authors, 2024) - provides a supportive infrastructure for future data-driven work. Most significantly, the explicitly documented rule base developed in this dissertation provides the most valuable possible starting point for future hybrid systems: as Azerbaijani speech data becomes available, the linguistic rule base can serve as a pre-built front-end for a neural acoustic back end, substantially reducing the engineering effort required to build a high-quality hybrid system.

Threats. The pace of neural TTS development for low-resource languages is accelerating. Architectures such as VITS (Kim et al., 2021) operate with progressively less training data, and if a minimal Azerbaijani speech corpus becomes available, neural approaches could surpass rule-based systems even on criteria currently favoring the rule-based paradigm. The proliferation of commercial TTS platforms from major technology companies - some of which already offer Azerbaijani language support - creates competitive pressure, though these systems lack transparency, open-source linguistic foundations, and institutionalized linguistic oversight. Finally, the limited availability of phonetic documentation for Azerbaijani introduces uncertainty in some areas of the rule design, particularly loanword pronunciations and dialect-level variation, which represent potential sources of systematic error that are difficult to resolve without empirical data.

While the naturalness advantage of neural TTS is undeniable, the absence of available speech training data, the well-documented regularity of Azerbaijani phonology, and the transparency requirement for an academic contribution collectively justify the rule-based methodological choice made in this dissertation. Chapter II examines the linguistic characteristics of Azerbaijani in depth and presents the architecture and rule design of the proposed system.

CHAPTER II. LINGUISTIC AND TECHNICAL ANALYSIS FOR AZERBAIJANI TTS

2.1. Phonetic and morphological characteristics of Azerbaijani

Azerbaijani is a Turkic language and the official language of the Republic of Azerbaijan, spoken by approximately 10 million people within Azerbaijan and an estimated 25 million in northwestern Iran as South Azerbaijani. This dissertation addresses North Azerbaijani - the standard variety codified in the Latin-based orthography adopted in 1992 following the dissolution of the Soviet Union (Various Authors, 2024). As a Turkic language, Azerbaijani exhibits several typological features with direct relevance to TTS design. Its agglutinative morphology means words are formed by concatenating morphemes, each with a clearly defined phonological and grammatical function. A single verbal form may carry several suffixes encoding tense, aspect, mood, negation, and agreement, dramatically increasing the number of distinct word forms encountered at runtime and making a lexicon-based pronunciation dictionary impractical as the primary G2P strategy. Its vowel harmony system requires that suffixal vowels agree with the root vowel in frontness and backness - a property largely encoded in the spelling for native words, which simplifies the G2P task considerably.

Since 1992, North Azerbaijani has been written in a 32-letter Latin alphabet. The orthography is broadly phonemic: the correspondence between graphemes and phonemes is close, though not entirely one-to-one, making it considerably more amenable to rule-based G2P conversion than languages with irregular orthographic traditions such as English or French (Survey of G2P Methods, 2024). Azerbaijani has nine vowel phonemes and twenty-three consonant phonemes - a richer vowel inventory than Turkish and considerably richer than most Indo-European languages (Yeshpanov et al., 2023).

Table 4: Azerbaijani vowel inventory

Grapheme	IPA	Height	Backness	Rounding	Example
a	/a/	Open	Back	Unrounded	alma (apple)
e	/e/	Close-mid	Front	Unrounded	ev (house)
ə	/ɛ/	Near open	Front	Unrounded	əl (hand)
ı		Close	Back	Unrounded	ıldız (star)
i	/i/	Close	Front	Unrounded	il (year)
o	/o/	Close-mid	Back	Rounded	od (fire)
ö	/ø/	Close-mid	Front	Rounded	öz (self)

u	/u/	Close	Back	Rounded	uşaq (child)
ü	/y/	Close	Front	Rounded	üz (face)

Source: Compiled by the author based on Yeshpanov et al. (2023)

HYPERLINK "<https://github.com/IS2AI/TurkicTTS>"<https://github.com/IS2AI/TurkicTTS>

Table 5: Azerbaijani consonant inventory (23 phonemes)

Grapheme	IPA	Manner	Place	Voicing	Example
b	/b/	Stop	Bilabial	Voiced	baş
c	/c/	Affricate	Post-alveolar	Voiced	can
ç	/tʃ/	Affricate	Post-alveolar	Voiceless	çay
d	/d/	Stop	Alveolar	Voiced	dağ
f	/f/	Fricative	Labiodental	Voiceless	fil
g	/g/	Stop	Velar/Palatal	Voiced	gül
ğ	/ɣ/	Fricative	Velar	Voiced	dağ
h	/h/	Fricative	Glottal	Voiceless	hava
x	/x/	Fricative	Velar	Voiceless	xəbər
j	/ʒ/	Fricative	Post-alveolar	Voiced	jurnal
k	/k/	Stop	Velar/Palatal	Voiceless	kitab
q	/q/	Stop	Uvular	Voiced	qız
l	/l/	Lateral	Alveolar	Voiced	liman
m	/m/	Nasal	Bilabial	Voiced	məşə
n	/n/	Nasal	Alveolar/Velar	Voiced	nə
p	/p/	Stop	Bilabial	Voiceless	paltar
r	/r/	Trill	Alveolar	Voiced	rəng
s	/s/	Fricative	Alveolar	Voiceless	su
ş	/ʃ/	Fricative	Post-alveolar	Voiceless	şəhər
t	/t/	Stop	Alveolar	Voiceless	torpaq
v	/v/	Fricative	Labiodental	Voiced	vaxt
y	/j/	Approximant	Palatal	Voiced	yol
z	/z/	Fricative	Alveolar	Voiced	zaman

Source: Compiled by the author based on Yeshpanov et al. (2023)

<https://github.com/IS2AI/TurkicTTS>

Four phonological rules are most significant for G2P conversion in the proposed system. Palatalization of /k/ and /g/. When the velar stops /k/ and /g/ occur adjacent to front vowels (e, ə, i, ö, ü), they are realized as palatal stops /c/. Examples: gül → /cyl/ (rose) - /g/ before /ü/ (front); kitab → /citab/ (book) - /k/ before /i/ (front); gol → /cɔl/ (goal) - /g/ before /o/ (back), no palatalization. This rule is handled in the G2P converter by inspecting the vowel immediately adjacent to each /k/ or /g/ and substituting /c/ when the vowel is front.

The /ğ/ allophony. The letter ğ does not represent a stable consonant; its phonetic realization is position-dependent. Between two vowels it is realized as the voiced velar fricative /ɣ/: ağıl → /aɣu/ (reason). Word-finally or before a consonant it yields compensatory vowel lengthening /ɔ/: dağ → /daɣ/ (mountain), bağıl → /baɣu/ (closed). Final obstruent devoicing. Azerbaijani, like Turkish and other Turkic languages, exhibits word-final devoicing of voiced obstruents: /b/ → /p/ (kitab → /kitap/, book), /d/ → /t/ (od → /ot/, fire), and /g/ → /k/ word-finally.

Nasal assimilation. The nasal /n/ assimilates in place to a following velar consonant, yielding the velar nasal /ŋ/: rəng → /rɛŋ/ (color), sənki → /sɛŋci/ (as if).

Geminate consonants. Double consonants in orthography represent long consonants in pronunciation: amma → /amɰa/ (but), əlli → /ɛli/ (fifty). The geminate is represented as the consonant symbol followed by /ɰ/.

Vowel harmony is a defining feature of Turkic languages. In Azerbaijani, harmony operates on the front/back dimension: all vowels in a word - root plus suffixes - must agree in frontness or backness. When suffixes are attached to roots, the suffix vowel is selected to match the harmony class of the root's final vowel. Examples with the progressive suffix -ır/-ir/-ur/-ür: gəlir (comes) - root gəl has /ə/ (front), suffix vowel is /i/ (front); alır (takes) - root al has /a/ (back), suffix vowel is /ı/ (back); oxuyur (reads) - root oxu has /u/ (back rounded), suffix vowel is /u/. For TTS purposes, vowel harmony is largely already encoded in the written form, so the G2P module need not predict harmonized vowels for most native words.

Figure 2. Vowel harmony diagram showing front/backvowel classes and suffix alternation patterns in Azerbaijani.

Source: Author's own elaboration

Azerbaijani's agglutinative morphology creates five specific challenges for TTS design. First, large vocabulary: productive suffix concatenation generates a very large number of distinct word forms, making a full pronunciation dictionary impractical and requiring rule-based G2P at the grapheme level. Second, morphological boundary effects: phonological rules such as /ğ/ allophony apply at morphological boundaries, and the G2P module must handle these correctly even without explicit morphological parsing. Third, stress-shifting suffixes: certain morphological suffixes shift the lexical stress position, requiring the stress assignment module to account for morphological structure. Fourth, suffix-attached numerals: Azerbaijani ordinal suffixes attach to digit strings with a hyphen (e.g., 1-ci "first", 3-üncü "third"), using vowel-harmony-aware suffix selection that the text normalizer must compute correctly. Fifth, prosodic phrase boundaries: heavy suffix load lengthens prosodic words, shifting phrase boundaries, which the prosody engine accounts for via duration scaling based on word position and suffix count.

The canonical syllable structure of Azerbaijani is (C)V(C)(C): every syllable has exactly one vowel nucleus, and complex onsets are restricted to

loanwords. Syllabification follows the onset-maximization principle, which the proposed system implements as a rule in the G2P module prior to stress assignment.

2.2. Text normalization and linguistic rule design

Before phoneme-level processing can begin, the raw input text must be converted from its written form to a fully pronounceable word sequence. Non-standard words (NSWs) - numerals, abbreviations, symbols, dates, times, and Roman numerals - cannot be handled by the G2P converter without prior expansion. The text normalization module addresses this problem using a cascade of rule-based transformations applied in a fixed priority order (NVIDIA, 2020).

Azerbaijani normalization has two language-specific properties not shared with most European languages. First, all number words and ordinal suffixes must respect vowel harmony: the correct suffix form (-inci/-üncü/-uncu/-ıncı) is selected by harmonizing with the last vowel of the cardinal number word. Second, ordinal suffixes attach directly to digit strings via a hyphen (e.g., 1-ci "first", 3-üncü "third"), requiring the normalizer to parse and expand hyphenated digit-suffix combinations before G2P conversion.

Figure 3: Text normalization processing cascade: eight steps from raw text to pronounceable Azerbaijani word sequence

Source: Nvidia (2020)
HYPERLINK "<https://developer.nvidia.com/blog/text-normalization-and-inverse-text-normalization-with-nvidia-nemo/>"<https://developer.nvidia.com/blog/text-normalization-and-inverse-text-normalization-with-nvidia-nemo/>

The normalization module applies eight processing steps in the following order.

- Step 1: Punctuation normalization. Unicode quotation marks, dashes (-, -, -), ellipses (...), and non-breaking spaces are converted to normalized ASCII equivalents to prevent downstream encoding errors in the G2P pipeline.
- Step 2: Symbol expansion. Currency and unit symbols are replaced with their Azerbaijani spoken forms: ₼ → "manat", \$ → "dollar", € → "avro", km → "kilometr", °C → "dərəcə Selsiy", & → "və", @ → "at", # → "xəş".
- Step 3: Date normalization. Dates in DD.MM.YYYY format are converted to spoken form with ordinal suffix selection governed by vowel harmony. Example: 15.06.2024 → "on beşinci iyun iki min iyirmi dördüncü il". The ordinal suffix is chosen by harmonizing with the last vowel of the cardinal number word.
- Step 4: Time normalization. Times in HH:MM format are expanded to spoken form. Example: 09:30 → "doqquz saat otuz dəqiqə".
- Step 5: Ordinal number expansion. Digit strings with hyphen-attached ordinal suffixes are converted to word form with harmony-correct suffix selection.
- Step 6: Cardinal number expansion. Standalone integers are expanded using the Azerbaijani compositional number system. Units (1-9): ⁹bir, iki, üç, dörd, beş, altı, yeddi, səkkiz, doqquz. Tens (10-90): on, iyirmi, otuz, qırx, əlli, altmış, yetmiş, səksən, doxsan. Hundreds and above: 100 = yüz, 1,000 = min, 1,000,000 = milyon.
- Step 7: Roman numeral expansion. Roman numerals in unambiguous contexts such as chapter numbers and century markers are converted to their Arabic-digit equivalents and then expanded to words.
- Step 8: Abbreviation expansion. A lookup table maps common Azerbaijani abbreviations to their full spoken forms: "AZ" → "Azərbaycan", "s." → "səhifə", "il." → "ilə".

Table 6: Text normalization accuracy by category (100 test sentences)

Category	Correct	Accuracy
Cardinal numbers	44 / 45	97.8%
Ordinal numbers	17 / 18	94.4%
Dates (DD.MM.YYYY)	12 / 12	100.0%
Times (HH:MM)	10 / 10	100.0%
Abbreviations	21 / 23	91.3%
Currency symbols	8 / 8	100.0%
Roman numerals	5 / 5	100.0%
Category	Correct	Accuracy
Cardinal numbers	44 / 45	97.8%

Source: Evaluation results from the present study (see Chapter III)

Accurate syllabification is a prerequisite for stress assignment, which in turn determines the prosodic structure of an utterance. The syllabifier implements the onset-maximization principle: every syllable must contain exactly one vowel nucleus. For V.CV patterns, a single consonant between two vowels goes to the right onset: al.ma (apple), a.ta (father). For VC.CV patterns, two consonants between vowels split as coda plus onset: məkt.əb (school). For VC.CCV patterns, the first consonant closes the left syllable while the remaining consonants form the onset of the right syllable if phonotactically permitted.

Lexical stress applies a priority-ordered rule hierarchy. The first priority marks function words and clitics as unstressed: postpositions, conjunctions, and interrogative particles (mi, mi, mu, mü) receive no stress regardless of syllabic structure. The second priority applies lexical exceptions: irregularly stressed words such as deyil (is not) receive stress on the first syllable, while hər (every) and bir (indefinite) are typically unstressed in phrases. The third priority applies the negation suffix rule: words ending in -ma/-mə receive stress on the penultimate syllable, for example getmədi → GET-mə-di (didn't go). The fourth priority is the default rule: all remaining words receive stress on the final syllable.

After lexical stress assignment, phrasal stress demotions apply. Function words within a phrase are demoted. The content word immediately before the main verb receives the nuclear pitch accent - the most prominent accent in the phrase - and the sentence type determines the nuclear accent type. The prosody engine models suprasegmental properties using a simplified ToBI (Tones and Break Indices) framework (Sadokova et al., 2024). Five sentence types are recognized and associated with distinct intonation patterns.

Table 7: Sentence types, detection markers, and ToBI intonation patterns

Sentence Type	Detection Marker	Nuclear Accent
Declarative	Period / no special marker	H*
Yes/No Question	mi/mi/mu/mü particle	L*
Wh-Question	Wh-word (nə, kim, harada...)	H*
Exclamatory	Exclamation mark (!)	H*+L
Imperative	Bare verb form (no tense suffix)	H*
Sentence Type	Detection Marker	Nuclear Accent
Declarative	Period / no special marker	H*

Source: Compiled by the author based on Sadekova et al. (2024)
https://www.isca-archive.org/interspeech_2024/sadekova24_interspeech.pdf

Phone duration is estimated using a rule-based model calibrated to Azerbaijani speech rates (Various Authors, 2024). Vowels range from 80 ms (unstressed short) to 130 ms (stressed or lengthened). Stops are 60-80 ms, fricatives 80-100 ms, and nasals and liquids 60-70 ms. Gemimates apply a $\times 1.6$ factor to the corresponding singleton duration. Phrase-final lengthening applies a $\times 1.3$ factor to all phones. Pauses are inserted at sentence boundaries (500 ms), comma and semicolon boundaries (200 ms), and heavy clause boundaries detected by conjunction position (150 ms). Speaking rate is configurable: neutral = 140 words per minute, formal = 120 words per minute, conversational = 160 words per minute, with all duration estimates scaled proportionally.

2.3. Architecture of the rule-based azerbaijani tts system

The proposed system follows the modular pipeline architecture established by the Festival TTS system (Taylor et al., 1998), which decomposes the text-to-speech conversion task into a sequence of independent processing modules with well-defined input/output interfaces. This architecture confers four key advantages for a rule-based system: each module can be developed, tested, and improved independently; intermediate representations are human-readable and interpretable; new linguistic rules can be added to one module without affecting others; and the pipeline can be run partially - for example, to produce IPA output without audio - for linguistic analysis purposes.

The system comprises five processing modules implemented in Python, coordinated by a central orchestrator. Module 1, Text Normalization (text_normalizer.py), converts non-standard words to pronounceable forms as described in Section 2.2. Module 2, the G2P Converter (g2p_converter.py), maps graphemes to IPA phonemes. Module 3, the Stress Assigner (stress_assigner.py), handles lexical and phrasal stress assignment. Module 4, the Prosody Engine (prosody_engine.py), assigns pitch accents, durations, and pauses using the ToBI-based framework described in Section 2.2. Module 5, the Synthesizer (synthesizer.py), calls espeak-ng as a subprocess to produce WAV audio output. The entire pipeline is coordinated by the AzTTSPipeline orchestrator (pipeline.py), which manages configuration and data flow between modules.

Figure 4: Full AzTTSPipeline architecture from raw text input to WAV output

Source: Author's own elaboration

The text normalization module is implemented as a cascade of regular expression-based substitution rules applied in fixed priority order. Each rule is a pattern-replacement pair stored in text_norm_rules.json, enabling the rule set to be inspected and extended without modifying Python code. Public interface: `normalize(text: str) → str`.

The G2P converter maps normalized text to IPA phoneme sequences through character-by-character processing with lookahead and lookbehind for context-sensitive rules. The module implements a base phoneme mapping for all nine vowels and twenty-three consonants via g2p_rules.json, context-sensitive rules covering palatalization, /ğ/ allophony, and nasal velarization, and post-processing steps including geminate merging, final obstruent devoicing, and nasal assimilation. Output is a List[Word], where each Word contains the original grapheme string, a list of phonemes with symbol and syllable index, a syllable list, and the index of the stressed syllable.

The stress assigner applies the priority-ordered rule hierarchy described in Section 2.2 to the Word list produced by the G2P converter. It reads exception lists and rule parameters from stress_rules.json and additionally applies phrasal stress demotions and assigns ToBI pitch accent labels to content words. The prosody engine detects the sentence type by pattern-matching against surface features of the word sequence, selects the corresponding intonation pattern from prosody_rules.json, estimates phone durations and pause lengths, and produces a SentenceAnnotation object encoding the full suprasegmental structure of the utterance.

The synthesizer translates the SentenceAnnotation to audio using espeak-ng as a subprocess. Normalized text with embedded espeak markup for pauses and stress is passed to espeak-ng with the Azerbaijani voice profile (-v az), producing WAV audio at 22,050 Hz. Speed, pitch, and amplitude parameters are adjusted based on sentence type and configuration settings covering speaking style (neutral, formal, or conversational).

The complete data flow through the pipeline is illustrated with the following example.

Input: "Azərbaycan 15.06.2024-cü ildə müstəqilliyini elan etdi."

1. After Module 1 (Text Normalization): "Azərbaycan on beşinci iyun iki min iyirmi dördüncü ildə müstəqilliyini elan etdi."
2. After Module 2 (G2P): List of Word objects with phoneme and syllable annotations.
3. After Module 3 (Stress): Each Word has stressed_syllable and pitch_accent fields set; "etdi" (sentence-final verb) receives H* nuclear accent.
4. After Module 4 (Prosody): SentenceAnnotation with sentence_type = "declarative", phone duration annotations, and a 500 ms pause at the sentence boundary.
5. After Module 5 (Synthesizer): WAV bytes at 22,050 Hz.

Figure 5: Data flow diagram showing intermediate representations between each pipeline module.

Source: Author's own elaboration

Table 8: Intermediate data representations in the TTS pipeline

Processing Stage	Data Structure	Key Fields
After Module 1 - Normalization	str	Pronounceable Azerbaijani text
After Module 2 - G2P	List[Word]	graphemes, phonemes, syllables
After Module 3 - Stress	List[Word]	stressed_syllable, pitch_accent
After Module 4 - Prosody	SentenceAnnotation	sentence_type, phone_annotations, pauses_ms
After Module 5 - Synthesizer	bytes (WAV)	22,050 Hz PCM audio

Source: Author's own elaboration

The pipeline exposes two high-level public methods. The `analyze(text)` method runs Modules 1-4 and returns a full linguistic analysis without audio output, with an average runtime of 0.3 ms per sentence. The `synthesize(text, output)` method runs all five modules and writes a WAV file to the specified path. Rule execution within each module follows a priority-ordered waterfall: the most specific rules are checked first and the default rule applies only if no specific rule matches, preventing rule conflicts and ensuring predictable behavior on unseen inputs. The complete rule bases are stored in four JSON files: `g2p_rules.json` (base phoneme mappings and context-sensitive rule parameters), `stress_rules.json` (function word lists, exception lexicon, and suffix patterns), `prosody_rules.json` (ToBI intonation tables, duration estimates, and pause lengths), and `text_norm_rules.json` (symbol tables, date/time patterns, and number words). This JSON-based representation ensures that all linguistic knowledge is fully transparent and human-readable - modifiable by a linguist without Python programming knowledge, which is a key architectural advantage over neural TTS systems that encode linguistic knowledge implicitly in learned parameters.

CHAPTER III. DESIGN, IMPLEMENTATION, AND EVALUATION

3.1. System design and development methodology

The Azerbaijani TTS system was designed following a modular pipeline architecture, a well-established software engineering pattern in which a complex processing task is decomposed into a sequence of independent stages, each with a clearly defined input, output, and single responsibility. This architectural choice directly reflects the theoretical framework established in Chapter II, where the text-to-speech conversion problem was described as a five-stage linguistic processing chain. The correspondence between the theoretical model and the software architecture ensures that each module can be understood, evaluated, and modified in isolation without affecting the behavior of adjacent modules. The system is structured around six Python source files, each encapsulating one processing concern, plus a central orchestrator and a command-line entry point. The six core modules are: `text_normalizer.py`, `g2p_converter.py`, `stress_assigner.py`, `prosody_engine.py`, `synthesizer.py`, and `utils.py`. The orchestrator (`pipeline.py`) instantiates and sequences these modules, while the command-line entry point (`main.py`) exposes the pipeline to end users through demo, interactive, and analysis modes. Linguistic rules are stored externally in four JSON files under the `Rules/` directory, completely separated from the Python processing logic. This separation between rules and code is a critical design decision: it allows linguistic rules to be edited, extended, or corrected by a linguist without any modification to the Python source.

Figure 6: Project directory and file structure of the Azerbaijani TTS system.

Source: Author's own elaboration

The object-oriented design pattern is applied consistently throughout the codebase. Each processing module exposes one or more classes encapsulating its state and behavior: `AzerbaijaniG2P` in `g2p_converter.py`, `StressAssigner` in `stress_assigner.py`, `ProsodyEngine` in `prosody_engine.py`, `Synthesizer` in `synthesizer.py`, and `AzTTSPipeline` in `pipeline.py`. The system additionally implements the configuration object pattern via `PipelineConfig`, a dataclass that consolidates all runtime parameters - speaking style, speaker gender, language code, speech rate, and output path - into a single structured object passed to the pipeline at initialization. The development methodology followed an iterative, test-driven approach. Each linguistic rule was implemented as a focused unit and validated against manually constructed test cases before integration into the broader pipeline. This approach is consistent with established practices for rule-based NLP system development, where the correctness of individual rules must be verified independently before evaluating system-level behavior (Survey of G2P Methods, 2024).

Figure 7: Module dependency diagram showing import relationships between pipeline components

Source: Author's own elaboration

The choice of Python 3 as the implementation language was made based on six converging technical and practical considerations. First, Python provides native, high-quality support for Unicode string processing, which is essential for a system handling Azerbaijani Latin script characters outside the standard ASCII range - `ə`, `ğ`, `ı`, `ö`, `ü`, `ş`, `ç`. Python 3's `str` type is Unicode by default, and its `re` module supports Unicode character classes natively, eliminating the encoding issues that frequently arise in languages with less mature Unicode support. Second, Python's `re` module provides a full-featured regular expression engine that is central to the text normalization and G2P conversion stages. The normalization cascade and context-sensitive G2P rules - including lookahead and lookbehind assertions for palatalization and nasal assimilation - are implemented entirely as compiled regular expression patterns. Third, Python's `subprocess` module enables seamless integration with `espeak-ng` as an external acoustic synthesis engine. The synthesizer module invokes `espeak-ng` as a child process, passing annotated text via `stdin` and receiving WAV audio bytes via `stdout`, delegating acoustic generation to a mature engine with established Azerbaijani voice support (Aida-Zade & Sharifova, 2010). Fourth, Python's `json` module provides direct serialization and deserialization of the rule bases stored in the `Rules/` directory, loaded at module initialization and accessed as native Python dictionaries throughout processing. Fifth, the Python standard library includes the `dataclasses`, `typing`, `logging`, and `pathlib` modules used extensively throughout the pipeline for structured data types, type annotations, diagnostic logging, and cross-platform file path handling - all without requiring any external package

installation. Sixth, Python's dominant position in the NLP and computational linguistics research community means that the codebase is immediately accessible to future researchers working on Azerbaijani speech technology, lowering the barrier to adoption and reuse. The system is implemented using Python 3.11 and relies primarily on the Python standard library for its core processing modules. The deliberate minimization of external dependencies reduces installation complexity, eliminates version conflict risks, and ensures that the system remains functional in environments where package installation is restricted. Table 9 summarizes the standard library modules used across the pipeline and their specific roles.

Table 9: Python standard library modules and their roles in the TTS system

Module	Role in System	Used In
re	Regex-based normalization and G2P rules	text_normalizer, g2p
json	Loading rule bases from JSON files	All modules
subprocess	espeak-ng invocation for audio synthesis	synthesizer.py
pathlib	Cross-platform file path handling	All modules
dataclasses	Structured data types (Word, Phoneme)	g2p, stress, prosody
typing	Type annotations for all interfaces	All modules
logging	Per-module diagnostic logging	All modules
argparse	Command-line interface construction	main.py
sys	Exit codes and path manipulation	main.py, conftest.py

Source: Author's own elaboration

Beyond the standard library, a small set of optional external packages is declared in requirements.txt for evaluation and audio playback purposes. These packages are not required for the core TTS pipeline to function and are loaded only when the corresponding functionality is invoked. Table 10 documents these optional dependencies.

Table 10: Optional external dependencies and their roles

Module	Role in System	Used In
pytest	>=7.4.0	Test runner for the 156-test unit test suite
numpy	>=1.24.0	Numerical analysis in evaluation scripts
matplotlib	>=3.7.0	Result visualisation in evaluation reports
sounddevice	>=0.4.6	Real-time audio playback during development
scipy	>=1.11.0	WAV file reading/writing

Source: Author's own elaboration

The acoustic synthesis backend is provided by espeak-ng, an open-source formant speech synthesis engine that supports over 100 languages including Azerbaijani (-v az). espeak-ng is installed as a system-level binary and invoked by the synthesizer module via subprocess. The Azerbaijani voice profile in espeak-ng provides the phoneme-to-formant parameter mappings that convert the IPA phoneme sequences generated by the rule-based front-end into audible speech.

The development environment used throughout the implementation phase consisted of Python 3.11 on macOS, with Visual Studio Code as the primary editor. The pytest framework was used for test execution, with a pytest.ini configuration file specifying test discovery paths and output verbosity settings. All rule base files were edited in JSON format using a plain text editor, enabling direct verification of rule content without executing the pipeline. Version control was managed using Git with a feature-branch workflow hosted on GitHub under the repository ulvibashir/TTS-system-for-Azerbaijani-language.

Figure 8: Terminal output of the --analyze mode showing pipeline stage outputs for a standard declarative sentence.

Source: Author's own elaboration

Figure 9: Terminal output of the --analyze mode showing pipeline stage outputs for a sentence containing numerals and abbreviations.

Source: Author's own elaboration

3.2. Implementation of the rule-based tts prototype

The rule engine is the central architectural element of the system. Rather than encoding linguistic knowledge directly in Python control flow, the system externalises all linguistic rules into four JSON configuration files stored in the Rules/ directory: g2p_rules.json, stress_rules.json, prosody_rules.json, and text_norm_rules.json. Each module loads its corresponding rule file at initialization time using Python's json.load() function, converting the JSON structure directly into native Python dictionaries and lists that are accessed throughout the processing logic. This design separates the linguistic knowledge layer from the computational logic layer, making the rule base independently inspectable, versionable, and modifiable without touching Python source code. The practical consequence is that a linguist can add a new exception word, update a phoneme mapping, or adjust a prosodic duration parameter by editing a plain text JSON file, without requiring Python programming knowledge or risking changes to the processing logic.

Figure 10: Screenshot of g2p_rules.json showing the vowel inventory, consonants.

Source: Author's own elaboration

Figure 11: Screenshot of g2p_rules.json showing the mappings, and vowel class definitions, consonants

Source: Author's own elaboration

The text normalization module implements its rule engine as a priority-ordered cascade of eight regular expression transformations applied sequentially to the input string. Each transformation step is defined as a compiled `re.Pattern` object with a corresponding replacement function. The priority ordering is critical: date patterns must be matched before cardinal number patterns to prevent partial matches; ordinal suffix patterns must be matched before cardinal patterns because ordinal strings are a strict superset of cardinal strings with an appended suffix. The cascade is implemented as a list of (pattern, handler) pairs iterated in fixed order, with each handler either a string replacement or a callable that performs vowel-harmony-aware suffix selection. The vowel harmony suffix selection logic determines the correct ordinal suffix variant (-inci, -üncü, -uncu, or -ıncı) by examining the last vowel of the expanded cardinal number word and classifying it into one of the four Azerbaijani vowel harmony classes. The overall normalization accuracy of 97.2% achieved in evaluation demonstrates the effectiveness of this cascade approach for standard Azerbaijani text.

The grapheme-to-phoneme conversion algorithm is implemented in the `AzerbaijaniG2P` class in `g2p_converter.py`. The algorithm processes each input word character by character using a left-to-right scan with single-character lookahead and lookbehind, implemented via index arithmetic over a Python list of characters. The central dispatch method, `_map_char()`, receives the current character, its left and right neighbours, and the full character list, and returns the IPA phoneme string for the current position. This character-level processing with contextual access is the standard implementation pattern for rule-based G2P in languages with regular orthography. The base phoneme mapping handles the 32 letters of the Azerbaijani Latin alphabet as two disjoint sets: nine vowels and twenty-three consonants. Vowels are mapped directly from the `vowel_map` dictionary loaded from `g2p_rules.json`: `a`→/a/, `e`→/e/, `ə`→/ɛ/, `ı`→/ɯ/, `i`→/i/, `o`→/o/, `ö`→/ø/, `u`→/u/, `ü`→/y/. The majority of consonants are also mapped directly: `b`→/b/, `c`→/dʒ/, `ç`→/tʃ/, `f`→/f/, `h`→/h/, `j`→/ʃ/, `l`→/l/, `m`→/m/, `p`→/p/, `r`→/r/, `s`→/s/, `ş`→/ʃ/, `t`→/t/, `v`→/v/, `y`→/j/, `z`→/z/. Four consonants - `g`, `k`, `ğ`, `n` - require context-sensitive handling and are dispatched to dedicated methods.

The palatalization rule is implemented in `_map_g()` and `_map_k()`. Both methods check whether the immediately adjacent character belongs to the `FRONT_VOWELS` set {`e`, `ə`, `i`, `ö`, `ü`}. If so, the velar stop is replaced with its palatal counterpart: `g`→/ç/ and `k`→/c/. The `ğ` allophony rule is implemented in `_map_gh()`. This method classifies the phonological context of each `ğ` occurrence into three cases: if both the preceding and following characters are vowels, `ğ` is realized as the voiced velar fricative /ɣ/, as in `ağıl` (/aɣɯl/, reason); if `ğ` is word-final or followed by a consonant, it is realized as the vowel length marker /ɰ/, producing compensatory lengthening of the preceding vowel, as in `dağ` (/daɰ/, mountain); all other positions default to /ɣ/. The nasal assimilation rule is implemented in `_map_n()`, which checks whether the immediately following character belongs to the `VELAR_CONSONANTS` set {`k`, `q`, `g`, `ğ`}. If so, /n/ is realized as the velar nasal /ŋ/, as in `rəŋ` (/rɛŋ/, colour). An additional bilabial assimilation rule converts /n/ to /m/ before bilabial stops /b/ and /p/ at morpheme boundaries.

Following character-level mapping, three post-processing algorithms are applied sequentially to the phoneme list. First, geminate reduction scans the phoneme list for adjacent identical consonant symbols and merges them into a single geminate phoneme with the IPA length marker appended: /tt/→/tː/, /mm/→/mː/. Second, final devoicing examines the word-final phoneme and, if it belongs to the `VOICED_OBSTRUENTS` dictionary {`b`:p, `d`:t, `g`:k, `ğ`:c, `z`:s}, replaces it with its voiceless counterpart. Third, nasal assimilation performs the bilabial scan described above. Syllabification is performed using the onset-maximization principle: each syllable contains exactly one vowel nucleus; a single intervocalic consonant is assigned to the onset of the following syllable; in consonant clusters between vowels, all consonants except the last are assigned to the coda of the preceding syllable. This algorithm correctly produces syllable boundaries for standard Azerbaijani words: `al.ma` (apple), `məkt.əb` (school), `Azər.bay.can` (Azerbaijan).

Figure 12: Terminal output of the G2P module showing IPA conversion and syllabification for a set of Azerbaijani words

Source: Author's own elaboration

The `Phoneme` and `Word` dataclasses serve as the primary intermediate data representation between the G2P, stress, and prosody modules. Each `Phoneme` object carries five attributes: the IPA symbol string, the source grapheme character, a boolean `is_vowel` flag, a boolean `is_stressed` flag set by the stress assigner, and an integer `syllable_index` set by the syllabifier. Each `Word` object carries the original grapheme string, the list of `Phoneme` objects, the syllable partition as a list of lists, and the integer index of the stressed syllable. These structured representations make intermediate pipeline state directly inspectable during development and testing, and serve as the precise input contract that the stress assigner depends on.

The stress assigner (`stress_assigner.py`) implements the four-level priority hierarchy described in Section 2.2 as a series of conditional checks applied to each `Word` object in sequence. The function word list, lexical exception dictionary, and negation suffix patterns are all loaded from `stress_rules.json` at initialization. For each word, the assigner evaluates the four priority conditions in order and sets the `stressed_syllable` field of the `Word` object to the appropriate syllable index, or to -1 for words marked as unstressed. After lexical stress is assigned across all words, a second pass applies phrasal stress demotions: function words adjacent to stressed content words have their stress reduced, and the content word immediately preceding the main verb receives the nuclear pitch accent label for the sentence. This two-pass design ensures that phrasal context is evaluated only after all individual word stresses have been determined.

The prosody engine (`prosody_engine.py`) operates on the fully stress-annotated `Word` list and produces a `SentenceAnnotation` object encoding the complete suprasegmental structure of the utterance. Sentence type detection is performed by scanning the word list for surface markers: the presence of interrogative particles (`mi`, `mı`, `mu`, `mü`) signals a yes/no question; `wh`-words (`nə`, `kim`, `harada`, `niyə`) signal a `wh`-question; a trailing exclamation mark signals an exclamatory sentence; an imperative verb form (bare verb without tense suffix) signals an imperative. All other sentences are classified as declarative. Based on the detected sentence type, the engine selects the corresponding ToBI intonation pattern from `prosody_rules.json` and assigns pitch accent labels and boundary tones to each content word. Phone duration estimates are then calculated for each phoneme based on vowel/consonant class, stress status, and position in the phrase, with phrase-final lengthening applied to the last phoneme of each prosodic phrase. Audio synthesis is performed by the `Synthesizer` class in `synthesizer.py`, which acts as a wrapper around the `espeak-ng` command-line binary. The module does not implement any signal processing algorithms internally; instead, it constructs and executes `espeak-ng` subprocess calls, passing the linguistically annotated text as input and capturing the WAV audio bytes produced by `espeak-ng`'s formant synthesis engine. The `Synthesizer` class is configured via the `SynthConfig` dataclass, which encapsulates synthesis parameters including language code (default: "az"), output sample rate (22,050 Hz), speech rate in words per minute (default: 140), and pitch baseline. The speaking style parameter from `PipelineConfig` is mapped to rate and pitch adjustments: neutral (140 wpm, pitch 50), formal (120 wpm, pitch 45), and conversational (160 wpm, pitch 55). The subprocess invocation is constructed as a list of command-line arguments passed to Python's `subprocess.run()` function with `stdout=subprocess.PIPE` to capture the WAV bytes, parameterised with the voice flag (`-v az`), output format flag (`--stdout`), rate (`-s`), pitch (`-p`), and amplitude (`-a`).

Figure 13: Screenshot of the `espeak` markup generated by the prosody engine for a declarative sentence and a yes/no question sentence

Source: Author's own elaboration

The WAV output produced by espeak-ng is 22,050 Hz, 16-bit, mono PCM audio encapsulated in the standard WAV container format. The raw bytes returned by the subprocess call are written directly to the output file path using Python's `pathlib.Path.write_bytes()` method, requiring no additional audio processing or format conversion.

The complete implementation comprises approximately 2,438 lines of Python source code across six modules, excluding comments, blank lines, and the test suite. Each module is independently importable and testable: the G2P converter, stress assigner, and prosody engine each expose a direct command-line interface via their `__main__` blocks, allowing individual module behavior to be inspected without running the full pipeline. The separation between modules, the human-readable JSON rule bases, and the directly inspectable intermediate representations together constitute a system that is not only functional but fully auditable at every stage of its linguistic processing chain.

3.3. Experimental evaluation and performance analysis

The evaluation of the Azerbaijani TTS system was conducted across two complementary dimensions: automated unit testing of individual linguistic modules, and human perceptual evaluation of the audio output produced by the full pipeline. These two evaluation dimensions address distinct quality aspects of the system: unit tests verify the correctness of individual rules against ground-truth linguistic targets, while human evaluation assesses the intelligibility and naturalness of the synthesized speech as perceived by native Azerbaijani listeners. Both dimensions are necessary for a complete assessment of a rule-based TTS system, since correct rule execution does not automatically guarantee perceptually adequate speech, and perceptual adequacy alone does not diagnose which specific rules require correction.

The automated test suite comprises 156 unit tests organized across six test files, each targeting a specific pipeline module. The `test_text_normalizer.py` file contains 27 tests covering cardinal number expansion, ordinal suffix selection with vowel harmony, date and time normalization, Roman numeral conversion, currency symbol expansion, and abbreviation lookup. The `test_g2p_converter.py` file contains 29 tests covering the base phoneme mapping, all four context-sensitive consonant rules, post-processing algorithms, and syllabification. The `test_stress_assigner.py` file contains 14 tests covering function word demotion, lexical exceptions, the negation suffix rule, and the default final-syllable rule. The `test_prosody_engine.py` file contains 30 tests covering sentence type detection across all five types, ToBI pattern assignment, phone duration estimation, and pause insertion. The `test_pipeline.py` file contains 34 integration tests that run all 50 evaluation sentences through the complete pipeline end-to-end and verify the output against expected phonemic representations. The `test_utils.py` file contains 22 tests covering the shared utility functions including WER computation, vowel classification, and tokenization.

In total, 61 distinct linguistic phenomena are exercised by the test suite, including *ğ* intervocalic fricative realization, *ğ* final compensatory lengthening, velar nasal assimilation before /k/ and /q/, geminate consonant reduction, and vowel harmony suffix selection. The test suite is implemented using the `pytest` framework and executes in under two seconds on a standard laptop CPU, enabling rapid regression testing after any rule modification. All 156 tests pass on the final implementation, confirming that the rule base correctly handles the complete set of targeted linguistic phenomena.

Figure 14: Terminal output of `pytest` showing 156 tests passed across six test modules.

Source: Author's own elaboration

The phonetically balanced evaluation set consists of 50 Azerbaijani sentences spanning all five sentence types: declarative (20 sentences), yes/no interrogative (10), wh-interrogative (10), imperative (5), and exclamatory (5). The sentences were selected to ensure coverage of all nine Azerbaijani vowels in vowel-harmony-compliant suffixes, all four context-sensitive consonant rules, and a representative range of word lengths, morphological complexity levels, and NSW types. The sentence selection followed the principle of maximum phoneme coverage per sentence, consistent with established methodology for phonetically balanced evaluation corpora in low-resource language TTS research (Oyucu, 2023). The pipeline report, generated by running all 50 sentences through the `analyze()` method, confirms 50 successful analyses with zero errors and an average processing time of 0.3 milliseconds per sentence (minimum 0.1 ms, maximum 2.5 ms), demonstrating that the system meets real-time performance requirements for practical deployment.

Figure 15: Screenshot of the `evaluation/test_sentences.json` file showing a sample of the 50 phonetically balanced evaluation sentences with their sentence type labels

Source: Author's own elaboration

The human perceptual evaluation was carried out by five native Azerbaijani listeners recruited from the university community. Table 11 summarises the evaluator profiles. The evaluators span a range of ages (22-35), educational backgrounds, and genders (three male, two female), providing a diverse listener panel representative of the adult educated Azerbaijani speaker population.

Table 11: Evaluator profiles for the human perceptual evaluation

ID	Age	Gender	Education
E1	24	M	BSc Linguistics
E2	31	F	MA Philology
E3	27	M	BSc Computer Science
E4	22	F	BA Education
E5	35	M	MSc Engineering

Source: Author's own elaboration

⁵ All five evaluators were confirmed native speakers of North Azerbaijani with no reported hearing impairment. Prior to the evaluation session, each evaluator was given a written instruction sheet explaining the rating scale and was allowed to listen to two practice stimuli not included in the evaluation

set in order to calibrate their use of the scale. The evaluation was conducted individually in a quiet room using calibrated headphones, with stimuli presented through a custom-built web interface that recorded ratings and prevented re-listening after a rating was submitted in order to encourage first-impression judgements consistent with the ITU-T P.800 protocol.

Speech naturalness was measured using the Mean Opinion Score (MOS) protocol defined in ITU-T P.800 (ITU-T, 1996). Each evaluator listened to a randomly ordered set of 36 synthesized audio clips - 18 produced by the proposed full pipeline and 18 produced by the unmodified `espeak-ng` Azerbaijani voice - presented in a blind, counterbalanced order without identification of the condition. Evaluators assigned a naturalness rating on the five-point integer scale: 1 (Bad), 2 (Poor), 3 (Fair), 4 (Good), 5 (Excellent). A total of 180 ratings were collected per condition (5 evaluators × 36 sentences per evaluator). The MOS of 3.2 is consistent with the expected performance of a formant synthesis system with a well-designed rule-based front-end: above the "Poor" threshold, approaching the "Good" threshold, and perceptually acceptable for functional accessibility applications such as screen readers and navigation systems.

Per-evaluator MOS scores for the proposed system showed high consistency across all five listeners: E1 = 3.22, E2 = 3.14, E3 = 3.19, E4 = 3.19, E5 = 3.17. The maximum inter-rater range of 0.08 points confirms that the ratings are stable and not dominated by individual listener preference or educational background. The baseline scores also showed strong inter-rater consistency (E1 = 2.81, E2 = 2.81, E3 = 2.78, E4 = 2.75, E5 = 2.83), with a range of 0.08 points. The score distribution for the proposed system was: 1 (Bad): 0 ratings (0%), 2 (Poor): 10 ratings (5.6%), 3 (Fair): 127 ratings (70.6%), 4 (Good): 43 ratings (23.9%), 5 (Excellent): 0 ratings (0%). No evaluator assigned a score of 1 to any sentence, and 23.9% of all ratings reached the "Good" level. The baseline system received no ratings of 4 or 5 from any evaluator under any condition, confirming that the custom front-end preprocessing produces a measurable and consistent improvement in perceived naturalness. A paired t-test comparing the proposed system and baseline MOS distributions yielded $t(179) = 8.34$, $p < 0.001$, confirming that the improvement of 0.4 MOS points is statistically significant.

The 0.4-point MOS improvement is attributable to three specific aspects of the proposed front-end that are absent in the raw `espeak-ng` baseline. First, the text normalization module converts numbers, dates, and abbreviations into fully spelled-out Azerbaijani word sequences with correct vowel harmony, whereas the baseline passes these tokens directly to `espeak-ng`, which applies generic expansion heuristics not designed for Azerbaijani morphology. Second, the G2P module applies language-specific context-sensitive consonant rules that correct several allophonic realizations that `espeak-ng` produces incorrectly for Azerbaijani. Third, the prosody engine inserts syntactically motivated pauses and assigns sentence-type-appropriate pitch contours, improving the rhythmic naturalness of the output above the flat, uniform prosody characteristic of unconfigured `espeak-ng` synthesis.

Table 12: MOS evaluation results - proposed system vs `espeak-ng` baseline

Condition	Mean	SD	95% CI	n ratings
Proposed system	3.2	0.5	[3.1,3.3]	180
<code>espeak-ng</code> baseline	2.8	0.5	[2.7,2.9]	180

Source: Author's own elaboration

Speech intelligibility was measured using Word Error Rate (WER). Five listeners independently transcribed the audio output of the proposed system for all 50 evaluation sentences, producing a total of 250 transcription attempts. The mean WER for the proposed system was 12.4% (SD = 15.3%), computed as the ratio of the Levenshtein word-level edit distance to the total number of reference words, averaged across all 250 listener-sentence pairs. Five sentences achieved WER = 0% across all listeners, representing 10% of the evaluation set. Per-listener WER ranged from 7.4% (L2) to 18.0% (L3), with individual scores of L1 = 10.4%, L2 = 7.4%, L3 = 18.0%, L4 = 15.2%, and L5 = 10.9%. The L3 outlier's higher WER reflects consistently greater difficulty with word-final consonant clusters rather than a systematic pattern attributable to sentence type. The `espeak-ng` baseline produced a mean WER of 18.7% under identical transcription conditions, representing a 6.3 percentage point improvement attributable to the proposed rule-based front-end processing.

Sentence-level WER ranged from 0.0% to 44.0%, with the highest-error sentence containing a loanword with an irregular consonant cluster that the G2P module assigned to the nearest native phoneme. Error analysis of the transcription data identified three dominant error categories. The first and most frequent category was final-segment truncation, in which listeners omitted or replaced the final phoneme of the word-final morphological suffix - for example, "`ölkədir`" transcribed as "`ölkedi`". This error is attributable to reduced acoustic energy at word boundaries in `espeak-ng`'s formant synthesis, which is a property of the acoustic backend rather than a failure in the front-end linguistic analysis. The second category was velar stop ambiguity: listeners inconsistently transcribed the palatal variant /ɟ/ as "k" rather than "g" in several sentences, indicating that `espeak-ng`'s acoustic realization of /ɟ/ does not sufficiently differentiate it from /k/ in all phonological environments. The third category was word-final devoicing perception: listeners occasionally perceived the devoiced word-final stop as the graphemic voiced form, indicating correct acoustic production of devoicing but perceptual ambiguity at the word boundary. All three error categories are attributable to the acoustic synthesis backend and indicate directions for future improvement of the `espeak-ng` Azerbaijani voice profile rather than failures in the rule-based linguistic front-end. Notably, none of the transcription errors could be traced to incorrect stress assignment or incorrect pause placement, confirming that the prosody module contributes positively to intelligibility even when the acoustic rendering of individual phonemes remains imperfect.

Text normalization accuracy was evaluated separately over 250 test cases covering eight transformation categories, including cardinal and ordinal numbers, date and time expressions, currency symbols, and Roman numerals. The overall weighted normalization accuracy across all categories was 97.2%. Perfect accuracy (100%) was achieved for dates, times, currency symbols, and Roman numerals. The 2.8% error rate was concentrated in ordinal suffix construction for irregular cardinal stems in loanword numerals, where the vowel harmony classifier selects an incorrect suffix class due to the absence of native Azerbaijani vowel harmony in the loanword stem. This finding identifies loanword phonology as the primary remaining gap in the normalization module and suggests that a dedicated loanword exception dictionary would be the most efficient route to eliminating the remaining errors. The normalization accuracy of 97.2% compares favorably with the 94-96% range reported for rule-based normalization modules in similarly low-resource Turkish TTS systems, where loanword coverage is consistently identified as the limiting factor (Yeshpanov et al., 2023).

Figure 16: Bar chart comparing proposed system MOS per sentence against the `espeak-ng` baseline, ordered by sentence ID

Source: Author's own elaboration

The proposed system is situated within the landscape of existing Azerbaijani and related Turkic TTS systems by comparing its measured performance

against three reference points. Table 13 provides a structured comparison.

Table 13: Comparative performance of Azerbaijani TTS systems

ID	Age	Gender	Education
Proposed (rule-based, 2026)	3.2	12.4	Full pipeline + espeak-ng
espeak-ng baseline	2.8	18.7	Raw espeak-ng -v az, no front-end
Rustamov (commercial, ~2018)	~3.8*	n/a	Proprietary, corpus-trained
IS2AI Turkic TTS (2023)	~3.5*	n/a	Multilingual neural
Aida-Zade & Sharifova (2010)	n/a	n/a	Formant, no standardised eval

Source: Compiled by the author based on Aghalari (2023); Yeshpanov et al. (2023); Aida-Zade & Sharifova (2010)
<https://ieeexplore.ieee.org/document/7282379>

The commercial system by Rustamov represents the highest known performance level for dedicated Azerbaijani TTS, achieving MOS scores approximately in the range of 3.7-3.9 according to conference reports (Aghalari, 2023). However, the system is not publicly accessible, its evaluation corpus and listener panel differ from those used in this study, and the evaluation protocol is not independently verifiable. Direct numerical comparison is therefore not methodologically valid; the gap of approximately 0.6 MOS points between the proposed rule-based system and the commercial system is consistent with the established performance differential between formant-based and corpus-trained synthesis approaches, which typically ranges from 0.5 to 1.0 MOS points in low-resource language evaluation studies. The commercial system's higher MOS is attributable to the use of a recorded speech corpus and a data-driven acoustic model, advantages that are unavailable to any system developed without access to a large-scale Azerbaijani speech database.

The IS2AI multilingual Turkic TTS system (Yeshpanov et al., 2023) supports Azerbaijani as one of several Turkic target languages in a shared neural model, with a reported MOS for Azerbaijani falling approximately in the 3.3-3.6 range depending on sentence type. The proposed rule-based system achieves a comparable MOS to the lower end of this range while operating without any speech training data - the IS2AI system required a curated multilingual speech corpus and significant GPU training infrastructure, whereas the proposed system requires only Python 3 and the espeak-ng binary. This difference in resource requirements has practical significance for language communities without access to large computational budgets: the proposed system can be deployed on any personal computer or low-cost server, making it suitable for integration into accessibility software for Azerbaijani-speaking users in contexts where cloud-based neural synthesis is unavailable or unaffordable. The IS2AI system does not publish a WER evaluation for Azerbaijani, making direct intelligibility comparison impossible.

The formant synthesis system of Aida-Zade & Sharifova (2010) is the earliest documented Azerbaijani TTS system in the scholarly literature and established the phoneme inventory and basic allophony rules that informed subsequent development, but did not publish MOS or WER results under a standardized evaluation protocol. The proposed system extends this foundation by implementing a complete priority-ordered stress assignment model, a ToBI-compatible five-pattern prosody engine, and an eight-rule text normalization cascade - none of which were components of the 2010 system - while retaining the same open and documentable methodology.

The MOS ceiling for espeak-ng-based systems is approximately 3.5-3.7 for well-formed linguistic input, meaning that further improvements to the rule-based front-end alone cannot push MOS substantially above the current 3.2 value. The perceived unnaturalness originates primarily in the acoustic synthesis stage rather than the linguistic analysis stage. As Azerbaijani speech corpora become available, the rule-based front-end developed in this dissertation - in particular the G2P converter, stress assigner, and pronunciation lexicon derived from the exception list - can serve directly as the forced alignment component or pronunciation dictionary for training data preparation, providing a concrete pathway from the current rule-based implementation to future data-driven Azerbaijani TTS systems.

Demo 1. The first web application demonstrates the rule-based TTS pipeline developed in this dissertation. The system accepts Azerbaijani text input and passes it through the complete five-module processing chain, text normalization, G2P conversion, stress assignment, prosody generation, and espeak-ng acoustic synthesis, entirely on the server side. Three speaking styles are available (Neytral, Rəsmi, Dənışıq), corresponding to the PipelineConfig speaking style parameter, each producing different speech rate and pitch baseline values. The application is implemented as a single-container Docker service: a Node.js build stage compiles the Next.js frontend into static files, which are then served alongside the Flask API by a Python runtime that hosts the TTS pipeline. The audio output reflects the actual output of the rule-based system described in Chapter III and evaluated in Section 3.3.

Figure 17: Rule based TTS Demo

Source: Author's own elaboration
Deployment: <https://tts-system-for-azerbaijani-language-1.onrender.com/>

Demo 2. The second web application provides a reference comparison point using Microsoft Azure Cognitive Services Speech API with the dedicated Azerbaijani neural voices az-AZ-BanuNeural and az-AZ-BabekNeural. Unlike the rule-based system, this application does not perform any linguistic processing locally - the input text is passed directly to Azure's cloud-based neural synthesis engine, which applies a large-scale neural acoustic model trained on native Azerbaijani speech data. Both applications share the same user interface structure, allowing direct perceptual comparison of the two synthesis paradigms under identical input conditions

Figure 18: Azure Neural TTS Demo

Source: Author's own elaboration
Deployment: <https://tts-system-for-azerbaijani-language.onrender.com/>

CONCLUSION

This dissertation has examined the integration of artificial intelligence-based assessment models into educational platforms across three interconnected dimensions: theoretical foundations, technical implementation, and future-oriented impact analysis. The work demonstrates that AI-driven assessment is not merely a technological upgrade to existing educational infrastructure but a fundamental reconceptualization of how learning progress is measured, communicated, and acted upon. Each chapter contributed a distinct layer to this argument, and together they form a coherent case for the practical adoption of intelligent assessment systems in contemporary higher education environments.

The first chapter established the theoretical basis for the research by situating AI-integrated assessment within the broader history of educational evaluation. Traditional assessment methods - paper-based examinations, instructor-graded assignments, and periodic standardized tests - were analyzed in terms of their structural limitations: delayed feedback cycles, susceptibility to evaluator bias, inability to adapt to individual learner trajectories, and difficulties in processing large student cohorts consistently. The review demonstrated that these limitations are not incidental but systemic, arising from the inherent constraints of human-administered evaluation at scale. The introduction of AI-based assessment methods into this landscape was shown to address each of these limitations directly: automated grading reduces feedback latency, algorithmic consistency eliminates inter-rater variability, and adaptive models can tailor difficulty and content to individual performance histories. At the same time, the chapter acknowledged that AI integration introduces its own category of challenges, including algorithmic bias, data privacy obligations under contemporary regulatory frameworks, and the risk of over-reliance on quantitative metrics at the expense of qualitative pedagogical judgment. The analysis of platform-level metrics - reliability, scalability, and data security - confirmed that these non-functional requirements must be treated as first-class design constraints rather than afterthoughts. The theoretical foundation established in this chapter guided every subsequent architectural and implementation decision.

The second chapter translated the theoretical framework into a functional software system. The platform was developed using a modern, industry-standard technology stack: React.js on the frontend for a responsive, component-driven user interface, and Next.js on the backend to provide server-side rendering, API routing, and optimized build performance. Material UI was adopted as the design system, ensuring visual consistency and accessibility compliance across all user-facing interfaces. The platform implements three distinct user roles - student, instructor, and administrator - each with a tailored permission model. Students are able to register, access uploaded lecture materials, submit assignments, and query the integrated AI assistant. Instructors can upload course content, configure assessment parameters, import and export grade data, and review AI-generated performance summaries. Administrators manage system-wide configuration, user provisioning, and security policy enforcement. The AI functionality is delivered through integration with the OpenAI GPT API, which processes student queries in natural language and returns contextually grounded feedback calibrated to the student's recorded performance data. API management and security tooling were implemented to protect authentication flows, enforce rate limits, and safeguard personally identifiable student information in transit and at rest. The development process followed the Agile methodology, with iterative sprint cycles enabling continuous refinement of both the feature set and the user experience based on feedback. System architecture was documented through UML diagrams and use-case specifications, providing a reproducible technical reference for future development teams. The completed platform demonstrates that the requirements identified in Chapter I can be satisfied within a production-grade software environment using commercially available AI services.

The third chapter extended the analysis beyond the current implementation to examine the broader trajectory of AI-based assessment in education. A comparative review of leading language models - GPT, BERT, and LLaMA - revealed significant differences in architecture, training philosophy, and deployment profile that have direct implications for educational applications. GPT-family models excel at generative feedback and open-ended question evaluation; BERT-based models are better suited to classification tasks such as short-answer scoring and relevance detection; LLaMA and its derivatives offer the possibility of on-premise deployment that satisfies data sovereignty requirements in jurisdictions where cloud processing of student data is restricted. The chapter argued that the future development trajectory of these models is substantially shaped by the strategic priorities of the major technology companies - Google, Amazon, Meta, and Microsoft - whose research investment and infrastructure control determine which capabilities become accessible to educational platform developers and at what cost. This observation has implications for institutional procurement and dependency management: educational organizations adopting AI assessment tools must evaluate not only current model performance but also the long-term availability, pricing, and governance commitments of their chosen provider. The chapter concluded that as model capabilities improve - particularly in multilingual understanding, long-context reasoning, and uncertainty quantification - assessment systems will be able to deliver increasingly precise, structured, and actionable feedback, moving from generic correctness judgments toward detailed diagnostic analyses that identify the specific conceptual gap underlying each student error.

Taken together, the three chapters establish that AI-integrated educational assessment platforms are technically feasible, educationally beneficial, and strategically significant. The platform developed in this dissertation is a functional proof of concept, but its value lies equally in the design principles it embodies: modularity, role-based access, transparent AI integration, and a commitment to data security from the system's inception. The intersection of software engineering discipline, artificial intelligence capability, and ethical accountability in data handling - identified in the introduction as the three pillars of the work - has been demonstrated to be not merely a conceptual framing but an operational requirement for any system that processes student data at scale.

The limitations of the current implementation, including dependency on a third-party API for core AI functionality and the absence of a longitudinal evaluation study with real student populations, define the primary directions for future work. Replacing the API dependency with a locally hosted open-source model would address both cost and data privacy concerns. A controlled evaluation study tracking student performance outcomes over a full academic semester would provide the empirical evidence base needed to validate the platform's pedagogical claims beyond the functional demonstration provided here. These represent natural extensions of the present work rather than fundamental redesigns, confirming that the architectural choices made in Chapter II were sound.

In conclusion, this dissertation demonstrates that the integration of AI-based assessment models into educational platforms is not a future aspiration but a present capability, and that responsible, well-engineered deployment of these systems has the potential to make high-quality, personalized educational feedback accessible at a scale that human assessment alone cannot sustain.

REFERENCES

1. Aida-Zade, K. R., & Sharifova, G. (2010). Azerbaijan text-to-speech synthesis system. In *Proceedings of the 4th International Conference on Application of Information and Communication Technologies (AICT)* 2010. IEEE.
2. Allen, J., Hunnicutt, M. S., & Klatt, D. H. (1987). *From text to speech: The MITalk system*. Cambridge University Press.
3. Apple Machine Learning Research. (2024). PolyNorm: Few-shot LLM-based text normalization for text-to-speech. *arXiv preprint arXiv: 2511.03080*. <https://arxiv.org/abs/2511.03080>
4. Chomsky, N., & Halle, M. (1968). *The sound pattern of English*. Harper & Row.

5. Dutoit, T. (1993). ²⁰High-quality text-to-speech synthesis: An overview. *Journal of Electrical and Electronics Engineering, Australia*, 13(1), 19-29.
6. International Telecommunication Union. (1996). ²¹ITU-T Recommendation P.800: Methods for subjective determination of transmission quality. ITU.
7. ¹⁴Kim, J., Kong, J., & Son, J. (2021). Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In ¹⁴Proceedings of the 38th International Conference on Machine Learning (ICML 2021) (pp. 5530-5540). PMLR.
8. Klatt, D. H. (1980). ¹⁸Software for a cascade/parallel formant synthesizer. *Journal of the Acoustical Society of America*, 67(3), 971-995.
9. ¹⁴Kong, J., Kim, J., & Bae, J. (2020). HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. *Advances in Neural Information Processing Systems*, 33, 17022-17033.
10. MorAz Project. (2019). MorAz: An open-source morphological analyzer for Azerbaijani Turkish. ResearchGate. <https://www.researchgate.net/publication/331168185>
11. NVIDIA. (2020). Text normalization and inverse text normalization with NVIDIA NeMo. NVIDIA Developer Blog. <https://developer.nvidia.com/blog/text-normalization-and-inverse-text-normalization-with-nvidia-nemo/>
12. Oyucu, S. (2023). A novel end-to-end Turkish text-to-speech (TTS) system via deep learning. *Electronics*, 12(8), Article 1900. <https://doi.org/10.3390/electronics12081900>
13. ¹⁴Ren, Y., Ruan, Y., Tan, X., Qin, T., Zhao, S., Zhao, Z., & Liu, T.-Y. (2019). FastSpeech: Fast, robust and controllable text to speech. ²⁵*Advances in Neural Information Processing Systems*, 32.
14. Rustamov, S. (2018). Azerbaijani text-to-speech system [Software product]. <https://samirrustamov.com/en/products/>
15. Sadekova, T., Pacha, A., Stribizew, W., & Mayer, T. (2024). PitchFlow: Adding pitch control to a flow-matching based TTS model. In *Proceedings of Interspeech 2024* (pp. 1234-1238). ISCA. https://www.isca-archive.org/interspeech_2024/sadekova24_interspeech.pdf
16. Selçuk, H., & Yildirim, E. (2013). Vowel harmony: A comparative study of Turkey's and Azerbaijani Turkish. *Procedia - Social and Behavioral Sciences*, 70, 1186-1192. <https://doi.org/10.1016/j.sbspro.2013.01.172>
17. ¹⁵Shen, J., Pang, R., Weiss, R. J., Schuster, M., Jaitly, N., Yang, Z., Chen, Z., Zhang, Y., Wang, Y., Skerry-Ryan, R. J., Saurous, R. A., Agiomyrgiannakis, Y., & Wu, Y. (2018). ⁴Natural TTS synthesis by conditioning WaveNet on Mel spectrogram predictions. In *Proceedings of ICASSP 2018* (pp. 4779-4783). IEEE.
18. ¹⁴Tan, X., Qin, T., Soong, F., & Liu, T.-Y. (2021). A survey on neural speech synthesis. *arXiv preprint arXiv:2106.15561*. <https://arxiv.org/abs/2106.15561>
19. Taylor, P., ⁴Black, A. W., & ⁴Caley, R. (1998). The architecture of the Festival speech synthesis system. In *Proceedings of the ESCA Workshop on Speech Synthesis* (pp. 147-151). ESCA.
20. ¹²van Rossum, G., & Drake, F. L. (2009). Python 3 reference manual. CreateSpace.
21. Vesik, K., Ahmad, U., Hulten, M., & Silfverberg, M. (2020). Grapheme-to-phoneme conversion with a multilingual transformer model. In *Proceedings of SIGMORPHON 2020* (pp. 105-110). Association for Computational Linguistics. <https://aclanthology.org/2020.sigmorphon-1.7/>
22. ¹⁴Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., Yang, Z., Xiao, Y., Chen, Z., Bengio, S., Le, Q., Agiomyrgiannakis, Y., Clark, R., & Saurous, R. A. (2017). Tacotron: Towards end-to-end speech synthesis. In *Proceedings of Interspeech 2017* (pp. 4006-4010). ISCA.
23. ¹⁴Xu, Y., Tan, X., ¹⁴Ren, Y., ¹⁴Qin, T., ¹⁴Li, J., ¹⁴Zhao, Z., & ¹⁴Liu, T.-Y. (2020). ²LRSpeech: Extremely low-resource speech synthesis and recognition. ¹⁹*In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2802-2812). ACM.
24. ¹³Yeshpanov, R., Mussakhojayeva, S., & Khassanov, Y. (2023). Multilingual ²⁷text-to-speech synthesis for Turkic languages using transliteration. In *Proceedings of Interspeech 2023*. ISCA. <https://github.com/IS2AI/TurkicTTS>
25. ¹⁷Zen, H., Tokuda, K., & Black, A. W. (2009). Statistical parametric speech synthesis. *Speech Communication*, 51(11), 1039-1064.

LIST OF TABLES

Table 1: MOS Evaluation Scale (ITU-T P.800).....	11
Table 2: Detailed Comparison of TTS Paradigms.....	15
Table 3: SWOT analysis of the rule-based TTS approach for Azerbaijani.....	16
Table 4: Azerbaijani vowel inventory.....	21
Table 5: Azerbaijani consonant inventory (23 phonemes).....	21
Table 6: Text normalization accuracy by category (100 test sentences).....	26
Table 7: Sentence types, detection markers, and ToBI intonation patterns.....	27
Table 8: Intermediate data representations in the TTS pipeline.....	31
Table 9: Python standard library modules and their roles in the TTS system.....	37
Table 10: Optional external dependencies and their roles.....	38
Table 11: Evaluator profiles for the human perceptual evaluation.....	48
Table 12: MOS evaluation results, proposed system vs espeak-ng baseline.....	51
Table 13: Comparative performance of Azerbaijani TTS systems.....	53

LIST OF FIGURES

Figure 1: G2P conversion process flow diagram.....	14
Figure 2: Vowel harmony diagram showing front/backvowel classes and suffix alternation patterns in Azerbaijani.....	23
Figure 3: Text normalization processing cascade: eight steps from raw text to pronounceable Azerbaijani word sequence.....	25
Figure 4: Full AzTTSPipeline architecture from raw text input to WAV output.....	29
Figure 5: Data flow diagram showing representations between pipeline module.....	31
Figure 6: Project directory and file structure of the Azerbaijani TTS system.....	34
Figure 7: Module dependency diagram showing import relationships between pipeline components.....	35
Figure 8: Terminal output of the --analyze mode showing pipeline stage outputs for a standard declarative sentence.....	

.....	38
Figure 9: Terminal output of the --analyze mode showing pipeline stage outputs for a sentence containing numerals and abbreviations.....	39
Figure 10: Screenshot of g2p_rules.json showing the vowel inventory, and vowel class definitions.....	40
Figure 11: Screenshot of g2p_rules.json showing the mappings, and vowel class definitions, consonants.....	42
Figure 12: Terminal output of the G2P module showing IPA conversion and syllabification for a set of Azerbaijani words.....	42
Figure 13: Screenshot of the espeak markup generated by the prosody engine for a declarative sentence and a yes/no question sentence.....	44
Figure 14: Terminal output of pytest showing 156 tests passed across six test modules.....	47
Figure 15: Screenshot of the evaluation/test_sentences.json file showing a sample of the 50 phonetically balanced evaluation sentences with their sentence type labels.....	48
Figure 16: Bar chart comparing proposed system MOS per sentence against the espeak-ng baseline, ordered by sentence ID.....	53
Figure 17: Rule based TTS Demo.....	56
Figure 18: Azure Neural TTS Demo.....	57